

컴퓨터 구조 이론 및 실습 보고서

<논리 게이트 및 조합논리회로>



동익대학교
DONG-EUI UNIVERSITY

학과 : 컴퓨터소프트웨어 공학과

과목 : 컴퓨터 구조 및 이론

학번 : 20212979

이름 : 임진호

목 차

1. 개념정리	3~16
2. Tinker cad를 이용한 시뮬레이션	17~35
3. 아두이노를 이용한 실습	36~54
4. 실습 요점, 결과, 검토사항	55~57

<개념이해>

1. 논리게이트

- 논리게이트 : 하나 또는 그 이상의 이진 입력에 대해 논리적인 연산을 수행하고 그 결과를 단일 이진 출력으로 나타냄
- 논리 회로 : 부울 대수를 이용하여 1개 이상의 논리 입력을 일정한 논리 연산에 의해 1개의 논리 출력을 얻는 회로

● AND 게이트 (7408)

- 논리곱 연산을 수행하는 논리게이트
- 모든 입력이 1인 경우에만 1출력, 나머지는 0을 출력

❖ AND 게이트

■ AND 게이트의 진리표

입력(A)	입력(B)	출력(X)
0	0	0
0	1	0
1	0	0
1	1	1

■ AND 게이트의 논리 기호



그림 3-1 AND 게이트의 논리 기호

■ AND 게이트의 논리식 표현

$$X = A \cdot B$$

■ AND 게이트의 집적회로

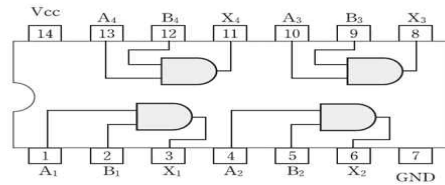


그림 3-2 단일 칩의 AND 게이트 집적회로(IC 7408)

● OR 게이트 (7432)

- 논리합 연산을 수행하는 논리게이트
- 최소한 하나 이상의 입력이 1인 경우 1이 출력

❖ OR 게이트

■ OR 게이트의 진리표

입력(A)	입력(B)	출력(X)
0	0	0
0	1	1
1	0	1
1	1	1

■ OR 게이트의 논리 기호



(a) 입력이 2개인 OR 게이트



(b) 입력이 3개인 OR 게이트

그림 3-4 OR 게이트의 논리 기호

■ OR 게이트의 논리식 표현

$$X = A + B$$

■ OR 게이트의 집적회로

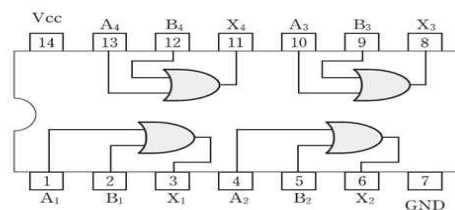


그림 3-5 단일 칩의 OR 게이트 집적회로(IC 7432)

● NOT게이트 (7432)

-입력 한 개와 출력 한 개인 게이트로 논리 부정을 나타낸다.

-입력 값의 반대가 출력된다.

❖ NOT 게이트

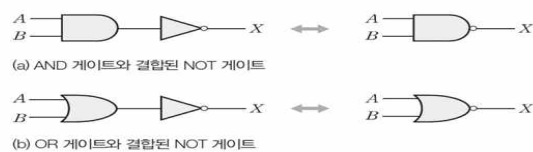
■ NOT 게이트의 진리표

입력(A)	출력(X)
0	1
1	0

■ NOT 게이트의 논리식 표현

$$X = \overline{A} = A'$$

■ NOT 게이트의 논리 기호



-NOT 게이트는 다른 게이트와 결합하는 경우도 많다.

● XOR 게이트 (7486)

-원래 명칭은 Exclusive OR이고, 배타적 OR로 해석한다.

-여러 개의 입력 중 1의 개수가 홀이면 1을 출력

-입력이 두 개인 XOR 게이트에서 두 입력이 서로 다를 때, 1을 출력하는 경우는 $A = 1, B = 0$ 또는 $A = 0, B = 1$ 이다

❖ XOR 게이트

■ XOR 게이트의 진리표

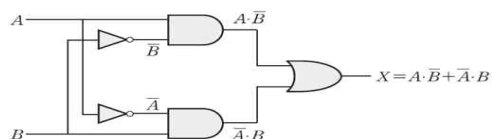
입력(A)	입력(B)	출력(X)
0	0	0
0	1	1
1	0	1
1	1	0

■ XOR 게이트의 논리식 표현

$$X = A \cdot \overline{B} + \overline{A} \cdot B$$

$$X = A \cdot \overline{B} + \overline{A} \cdot B = A \oplus B$$

■ XOR 게이트의 논리 기호



● NAND게이트 (7400)

-AND 게이트와 NOT 게이트를 결합한 것이다. 따라서 AND 게이트의 결과를 반대로 출력한다.

-모든 입력이 1인 경우만 0을 출력하고, 나머지 경우는 1을 출력.

❖ NAND 게이트

■ NAND 게이트의 진리표

입력(A)	입력(B)	출력(X)
0	0	1
0	1	1
1	0	1
1	1	0

■ NAND 게이트의 논리 기호

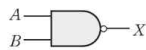


그림 3-15 NAND 게이트의 논리 기호

■ NAND 게이트의 논리식 표현

$$X = \overline{A \cdot B}$$

■ NAND 게이트의 집적회로

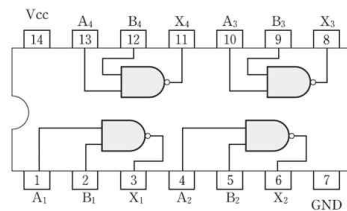


그림 3-16 단일 칩의 NAND 게이트 집적회로(IC 7400)

● NOR 게이트 (7402)

-OR 게이트와 NOT 게이트의 결합. OR 게이트의 결과에 반대값을 출력.

-다수의 입력 중 최소 하나 이상의 입력이 1인 경우 0을 출력

-NOR게이트는 직접회로에서 입,출력의 방향이 반대인 점을 유의해야한다.

❖ NOR 게이트

■ NOR 게이트의 진리표

입력(A)	입력(B)	출력(X)
0	0	1
0	1	0
1	0	0
1	1	0

■ NOR 게이트의 논리 기호



그림 3-18 NOR 게이트의 논리 기호

■ NOR 게이트의 논리식 표현

$$\begin{aligned} \text{입력이 2개일 때} \quad & X = \overline{A + B} \\ \text{입력이 3개일 때} \quad & X = \overline{A + B + C} \end{aligned}$$

■ NOR 게이트의 집적회로

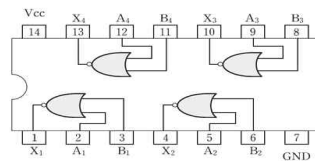


그림 3-19 단일 칩의 NOR 게이트 집적회로(IC 7402)

● 범용 논리 게이트

-NAND 게이트와 NOR게이트를 이용하여 디지털 시스템에서 사용되는 모든 게이트를 구성할 수 있다.

-NAND 게이트와 NOR 게이트를 유니버설 게이트 또는 범용 게이트 라고한다.

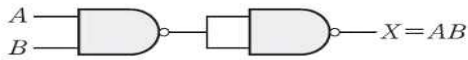


그림 3-25 NAND 게이트로 구성된 AND 게이트

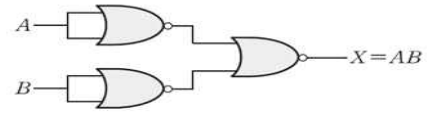


그림 3-26 NOR 게이트로 구성된 AND 게이트

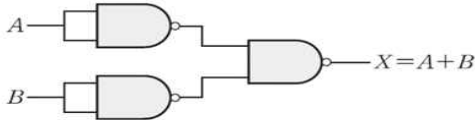


그림 3-27 NAND 게이트로 구성된 OR 게이트

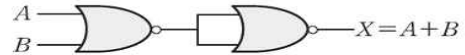


그림 3-28 NOR 게이트로 구성된 OR 게이트

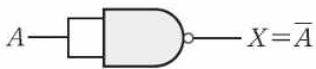


그림 3-29 NAND 게이트로 구성된 NOT 게이트

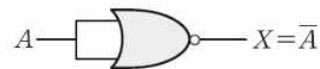


그림 3-30 NOR 게이트로 구성된 NOT 게이트

● 부울 대수의 기본 법칙

- 부울 대수 : 2진수와 논리 동작을 취급하는 대수로, 0과 1만의 값을 취하는 논리회로의 형태와 구조를 기술하는 데 필요한 수학적 이론이다.
- 부울 대수를 사용하면 동일한 성능을 갖는 더 간단한 회로를 만들 수 있다.

※ 교환법칙

- 입력 순서가 변경되더라도 논리연산의 결과는 동일하게 출력.

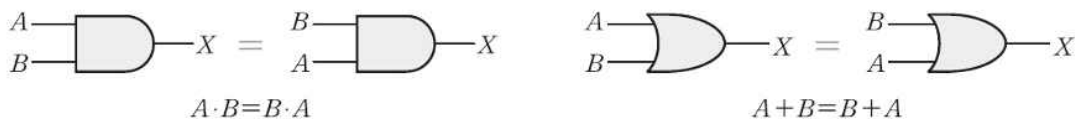


그림 3-31 교환 법칙이 성립하는 등가의 논리회로

A	B	$A \cdot B$	$B \cdot A$	$A + B$	$B + A$
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	0	1	1
1	1	1	1	1	1

진리표를 통해 AND 연산과 OR 연산에서 교환 법칙이 성립함을 알 수 있다.

※ 결합법칙

-3개의 입력이 동일한 논리연산을 수행할 때, 입력의 순서가 바뀌어 연산이 수행되어도 결과는 동일하게 나타난다.

-AND와 OR의 연산 순서가 괄호 표시에 의해 변경되어도 결과는 동일하다.

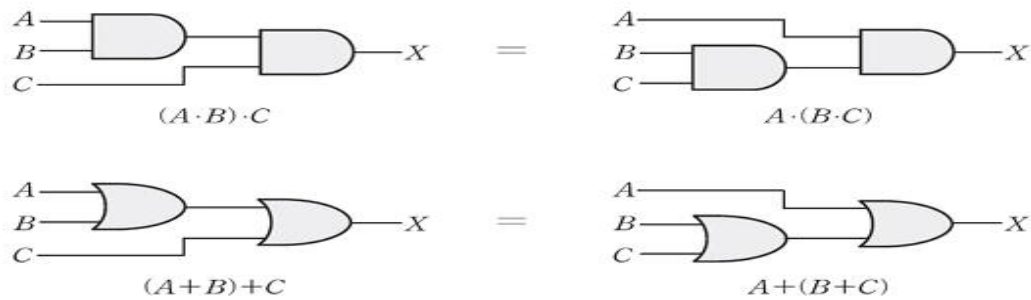


그림 3-32 결합 법칙이 성립하는 등가의 논리회로

-진리표를 통해 입력 A, B, C의 AND,OR연산에서 결합법칙이 만족하는 것을 알 수 있다.

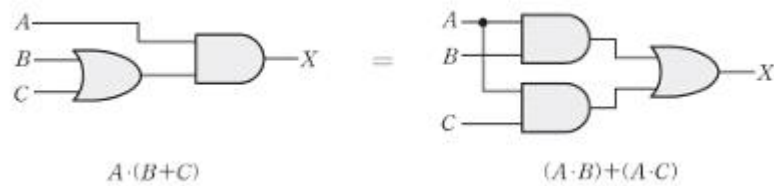
A	B	C	$(A \cdot B) \cdot C$	$A \cdot (B \cdot C)$	$(A + B) + C$	$A + (B + C)$
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	0	0	0	1	1
0	1	1	0	0	1	1
1	0	0	0	0	1	1
1	0	1	0	0	1	1
1	1	0	0	0	1	1
1	1	1	1	1	1	1

※ 분배 법칙

-입력이 3개일 때 입력B,C로 OR연산을 수행한 후 그 결과와 입력A를 AND연산하는 것과, 입력B와 C를 각각 입력 A와 AND 연산한 후 두 결과로 OR 연산을 수행하는 것이 동일하다.

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + B \cdot C = (A + B) \cdot (A + C)$$



-입력이 2개인 AND 게이트와 OR게이트로 입력이 3개인 논리회로를 만들어 분배 법칙이 성립하는 것을 보여준다.

A	B	C	$A \cdot (B+C)$	$(A \cdot B) + (A \cdot C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

-입력 A, B, C에 대한 불 논리식의 진리표는 분배 법칙을 만족하는 것을 보여준다.

※다중 부정

-논리 부정, 즉 NOT 연산을 여러 번 수행하는 것이다.

-만약 논리 부정을 두 번 연속 수행하면 원래 자신이 된다.

$$\overline{\overline{A}} = A$$

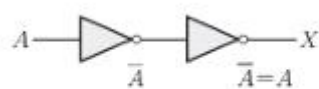


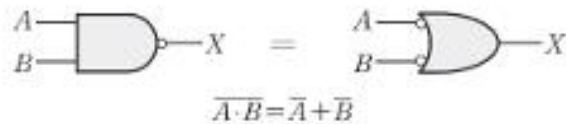
그림 3-34 NOT 게이트를 이용한 이중 부정

A	$\overline{A}$	$\overline{\overline{A}}$	$\overline{\overline{\overline{A}}}$
0	1	0	1
1	0	1	0

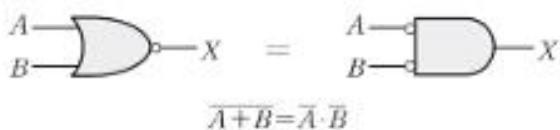
-NOT 게이트 2개를 직렬로 연결하여 원래 입력 A 값을 얻는 것을 알 수 있다.

● 드모르간의 법칙

-여러 논리 변수의 논리합 전체를 부정하면 원래 논리 변수 각각을 논리곱 한 것과 같다.



-여러 논리 변수의 논리곱 전체를 부정하면 원래 논리 변수 각각을 부정하여 논리합 한 것과 같다.



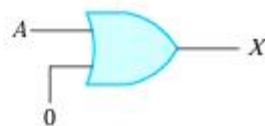
-드모르간의 법칙이 성립함을 보여주는 진리표이다.

A	B	$\overline{A}$	$\overline{B}$	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$	$\overline{\overline{A} + \overline{B}}$	$\overline{A \cdot B}$
0	0	1	1	1	1	1	1
0	1	1	0	1	1	0	0
1	0	0	1	1	1	0	0
1	1	0	0	0	0	0	0

● 부울 대수의 기본 정리 12가지

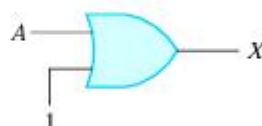
1. $A+0 = A$

A	0	$A+0=A$
0	0	$0+0=0$
1	0	$1+0=1$



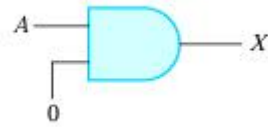
2. $A+1 = 1$

A	1	$A+1=1$
0	1	$0+1=1$
1	1	$1+1=1$



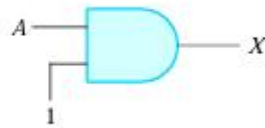
3. $A \cdot 0 = 0$

A	0	$A \cdot 0 = 0$
0	0	$0 \cdot 0 = 0$
1	0	$1 \cdot 0 = 0$



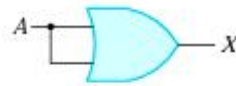
4. $A \cdot 1 = A$

A	1	$A \cdot 1 = A$
0	1	$0 \cdot 1 = 0$
1	1	$1 \cdot 1 = 1$



5. $A + A = A$

A	A	$A + A = A$
0	0	$0 + 0 = 0$
1	1	$1 + 1 = 1$



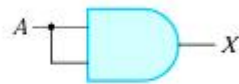
6. $A + A' = 1$

A	$\bar{A}$	$A + \bar{A} = 1$
0	1	$0 + 1 = 1$
1	0	$1 + 0 = 1$



7. $A \cdot A = A$

A	A	$A \cdot A = A$
0	0	$0 \cdot 0 = 0$
1	1	$1 \cdot 1 = 1$



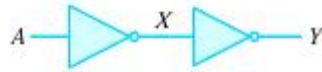
8. $A \cdot A' = 0$

A	$\bar{A}$	$A \cdot \bar{A} = 0$
0	1	$0 \cdot 1 = 0$
1	0	$1 \cdot 0 = 0$



9. $A'' = A$

A	$\bar{A}$	$\bar{\bar{A}} = A$
0	1	0
1	0	1



10. $A + A \cdot B = A$

A	B	$A \cdot B$	$A + A \cdot B$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

- 분배 법칙을 이용한 후 기본 정리 2와 4를 적용하면 다음과 같은 결과를 얻는다.

$$\begin{aligned}
 A + A \cdot B &= A \cdot (1 + B) && \text{분배 법칙} \\
 &= A \cdot 1 && \text{기본 정리 2 } (1 + B = 1) \\
 &= A && \text{기본 정리 4 } (A \cdot 1 = A)
 \end{aligned}$$

11. $A + A' \cdot B = A + B$

A	B	$\bar{A}$	$\bar{A} \cdot B$	$A + \bar{A} \cdot B$	$A + B$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

$$\begin{aligned}
 A + \bar{A} \cdot B &= A + A \cdot B + \bar{A} \cdot B \\
 &= A \cdot A + A \cdot B + \bar{A} \cdot B \\
 &= A \cdot A + A \cdot B + A \cdot \bar{A} + \bar{A} \cdot B \\
 &= (A + \bar{A}) \cdot (A + B) \\
 &= 1 \cdot (A + B)
 \end{aligned}$$

12. $(A + B) \cdot (A + C) = A + B \cdot C$

A	B	C	$A+B$	$A+C$	$B \cdot C$	$(A+B) \cdot (A+C)$	$A + B \cdot C$
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	0	1	1
1	0	1	1	1	0	1	1
1	1	0	1	1	0	1	1
1	1	1	1	1	1	1	1

-부울 대수의 기본정리.

㉑ $A+0=A$	㉒ $A+1=1$	㉓ $A \cdot 0=0$
㉔ $A \cdot 1=A$	㉕ $A+A=A$	㉖ $A+\bar{A}=1$
㉗ $A \cdot A=A$	㉘ $A \cdot \bar{A}=0$	㉙ $\bar{\bar{A}}=A$
㉚ $A+A \cdot B=A$	㉛ $A+\bar{A} \cdot B=A+B$	㉜ $(A+B) \cdot (A+C)=A+B \cdot C$

● 카르노 맵을 통한 논리함수 간략화

-카르노 맵 : 부울 대수식을 간소화하기 위한 가장 체계적이고, 간단한 방법이며, 입력 변수의 2진 값을 나타낼 수 있는 셀들을 적절히 묶어 최적의 간략화된 부울 대수식을 얻을 수 있다.

-변수가 n개이면, 2^n 가지의 입력 조합을 도표로 표현할 수 있다.

A	B	C	X	최소항	기호
0	0	0	1	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	m_0
0	0	1	1	$\bar{A} \cdot \bar{B} \cdot C$	m_1
0	1	0	0	$\bar{A} \cdot B \cdot \bar{C}$	m_2
0	1	1	0	$\bar{A} \cdot B \cdot C$	m_3
1	0	0	0	$A \cdot \bar{B} \cdot \bar{C}$	m_4
1	0	1	1	$A \cdot \bar{B} \cdot C$	m_5
1	1	0	0	$A \cdot B \cdot \bar{C}$	m_6
1	1	1	1	$A \cdot B \cdot C$	m_7

-임의의 출력값을 갖는 3 변수의 대한 진리표이다.

-출력이 1인 것끼리 모아 간략화 한 것을 곱의 합 형식이라고 하고, 출력이 0끼리 간략화한 것을 합의 곱 형식이라고 한다.

A \ BC	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$ m0	$\bar{A}\bar{B}C$ m1	$\bar{A}B\bar{C}$ m3	$\bar{A}BC$ m2
1	$A\bar{B}\bar{C}$ m4	$A\bar{B}C$ m5	$AB\bar{C}$ m7	ABC m6

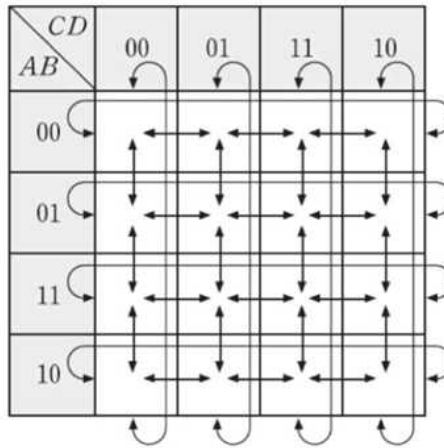
C \ AB	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$A\bar{B}\bar{C}$	$AB\bar{C}$
1	$\bar{A}B C$	$A\bar{B} C$	$AB C$	$A\bar{B} C$

-입력의 일반적 순서는 00, 01, 10, 11순서이지만 카르노 맵에서는 00, 01, 11, 10이다. 이로 인해 셀을 기호로 표현할 때 순차적이지 못 함

A \ BC	00	01	11	10
0	1	1		
1		1	1	

C \ AB	00	01	11	10
0	1			
1	1		1	1

-출력이 0인 경우에는 빈칸으로 두고, 출력이 1인 경우에만 1을 표시한다.



- 카르노 맵에서 행과 열의 이웃 관계를 이용하여 부울 대수식을 간략화할 수 있다.
- 먼저, 주어진 부울 대수식 또는 진리표를 참고해 카르노 맵을 작성한다.
- 두 번째로는, 1로 표시된 이웃들을 1, 2, 4, 8, 16개씩 그룹화한다. 이때 가능하면 큰 개수로 그룹화하는 것이 좋으며, 각각 다른 그룹에 여러 번 중복하여 그룹화가 가능하다.
- 세 번째로는, 그룹화를 시킨 그룹을 간략화한다.
- 마지막으로 간략화된 부울 대수식끼리 OR 연산을 한다.

$AB \backslash CD$	00	01	11	10
00				
01				
11	1	1		
10				

$X = ABC$

$AB \backslash CD$	00	01	11	10
00	1			1
01				
11				
10				

$X = \overline{A}BD$

$AB \backslash CD$	00	01	11	10
00	1			
01				
11				
10	1			

$F = \overline{B}CD$

< 행과 열의 이웃 관계를 이용해 그룹화 한 예시 >

● 무관 조건(don't care)

-무관 조건 : 출력에 관여하지 않는 입력 변수이다.

-이웃 영역을 그룹화 할 때 가장 간단한 표현을 얻기 위해 임의로 채울 수 있다.

-간략화 과정에서 그룹화할 수도, 하지 않을 수도 있다.

CD \ AB	00	01	11	10
00	1			
01	1			
11	1			
10				

(a) 무관 조건을 사용하지 않은 경우

CD \ AB	00	01	11	10
00	1			
01	1	X		
11	1			
10	X			

(b) 무관 조건을 사용한 경우

>> 무관 조건 활용 예시

● 조합 논리회로§

-조합 논리회로 : 입력과 출력이 있는 논리 게이트 집합으로 출력은 현재 입력에 의해 결정되며, 순차 논리회로와 비교해 기억 능력이 없다.

○ 가산기- 두 개 이상의 입력을 이용하여 이들의 합을 출력하는 조합 논리회로이다.

- 종류는 반가산기, 전가산기가 존재한다.

○반가산기

- 1비트를 사용하는 두 개의 입력과 두 개의 출력으로 합(sum)과 자리 올림(carry)이 사용된다.

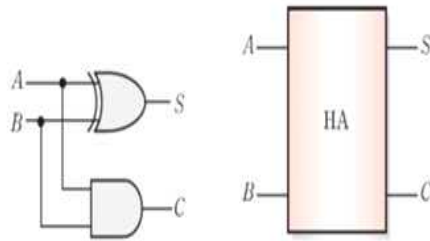
- 자리 올림수(C)는 A와 B의 값이 모두 1일 때, 합(S)은 A와 B의 값이 서로 다를 때만 발생한다.

- 2진수 1비트만을 덧셈 연산하기 때문에 하위 비트에서 발생하는 자리 올림을 고려하지 않는다.

- 반가산기를 카르노 맵으로 표현했을 때 자리 올림(C)에 대해서는 $C = A \cdot B$ (AND 게이트)로 표현할 수 있고, 합(S)에 대해서는 $S = A \cdot B' + A' \cdot B = A \oplus B$ (XOR 게이트)로 표현할 수 있다.

- 반가산기의 진리표, 논리 회로, 논리 기호는 아래와 같다

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



● 전가산기

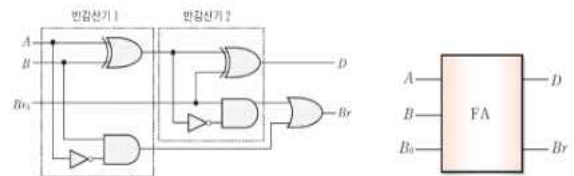
- 두 입력, 2진수 A와 B 그리고 하위 비트 올림수(C0)를 포함하여 2진수 세 개를 더하는 조합 논리회로이다.
- 전가산기의 논리회로는 반가산기 두 개가 결합한 형태로 구성되어있다.
- 아래의 카르노 맵을 통해 자리 올림수에 대한 간략화 결과는 $C = A \cdot C_0 + A \cdot B + B \cdot C_0$ 이라는 것을 알 수 있고, 합에 대한 간략화 결과는 $S = A \oplus B \oplus C_0$ 이라는 것을 알 수 있다.

BBr_0 A	00	01	11	10
0		1	1	1
1			1	

(a) 빌림수

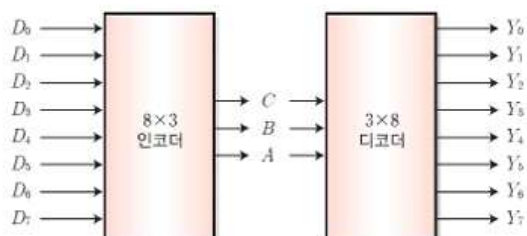
BBr_0 A	00	01	11	10
0		1		1
1	1		1	

(b) 차



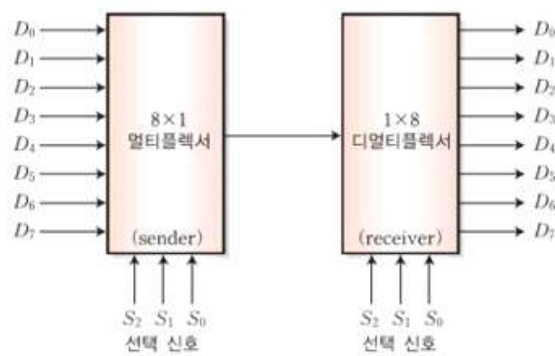
● 인코더와 디코더

- 인코더 : 외부에서 들어오는 임의의 신호를 부호화된 신호로 변환하여 컴퓨터 내부로 들여보내는 조합 논리회로로, 2^n 개의 입력신호를 n개의 출력신호로 만든다. 2^n 개의 비트 중 오직 한 비트만이 활성화된 1의 값을 갖는다.
- 디코더 : n비트의 2진 코드를 최대 2^n 가지의 정보로 바꿔주는 조합 논리회로이다. 다수의 입력신호로 1개의 출력신호를 얻는다. 디코더와는 반대되는 동작을 한다.
- 인코더와 디코더의 관계



● 멀티플렉서와 디멀티플렉서

- 멀티플렉서 : 여러 개의 입력 중 하나만 출력에 전달하는 조합 논리회로이다.
- 디멀티플렉서 : 한번에 들어온 여러 신호 중 하나를 골라 출력하는 조합 논리회로이며, 멀티플렉서와 반대되는 기능을 수행하는 회로이다.
- 멀티플렉서와 디멀티플렉서의 관계

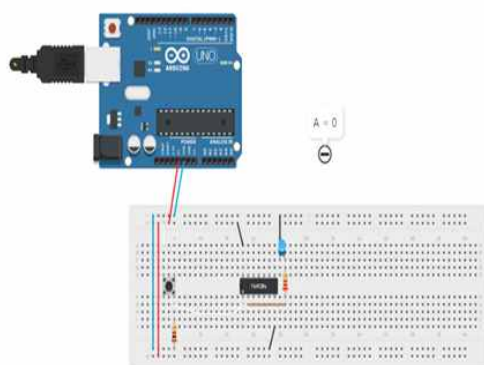


<Tinker Cad를 이용한 회로 시뮬레이션>

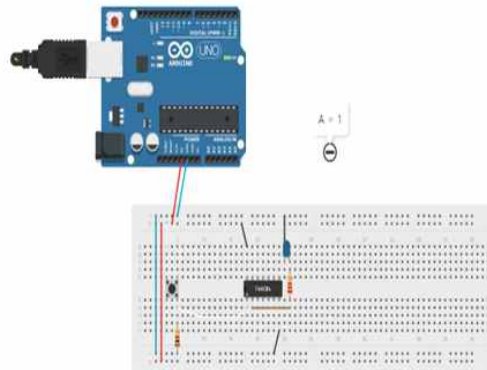
(Tinker cad를 이용하여 회로 제작 및 시뮬레이션 실행)

▶ NOT 게이트

1. $A = 0$



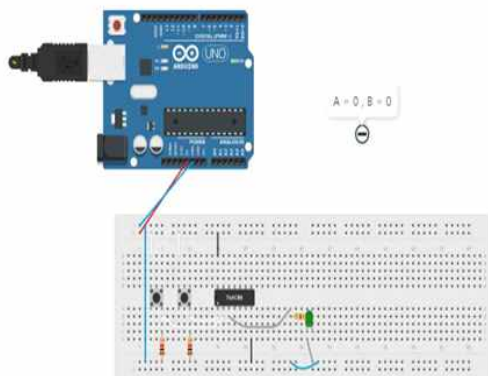
2. $A = 1$



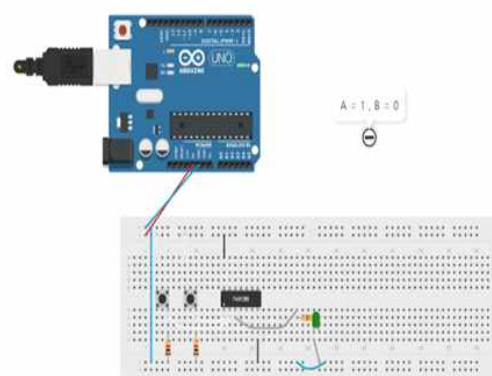
▶ AND 및 NAND 게이트

-AND 게이트

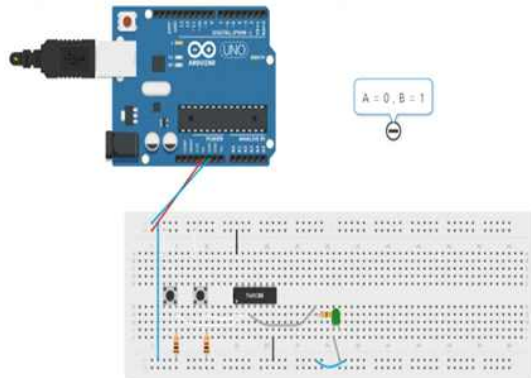
1. $A = 0, B = 0$



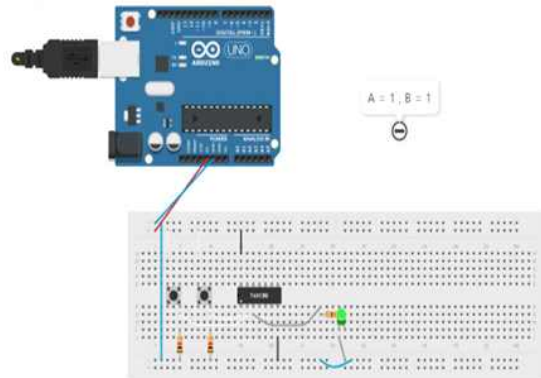
2. $A = 1, B = 0$



3. $A = 0, B = 1$

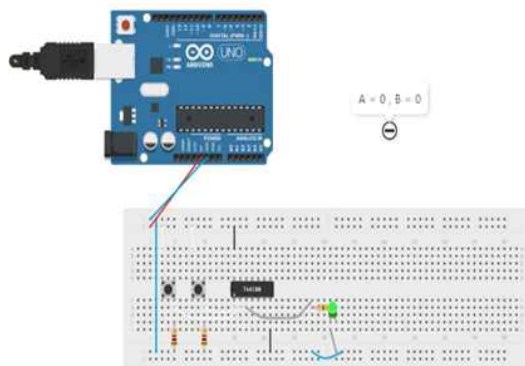


4. $A = 1, B = 1$

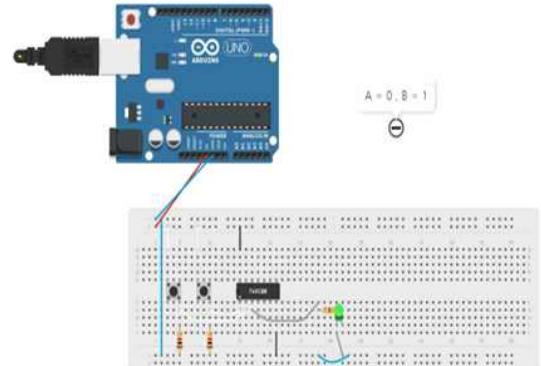


-NAND 게이트

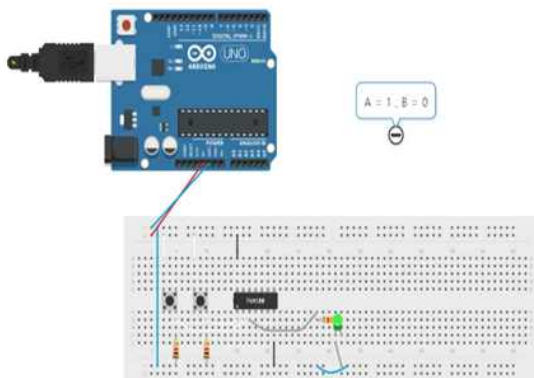
1. $A = 0, B = 0$



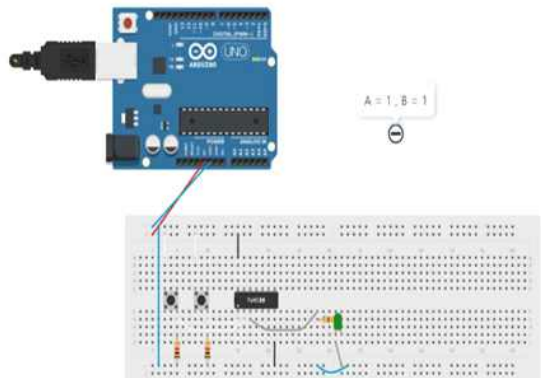
2. $A = 0, B = 1$



3. $A = 1, B = 0$



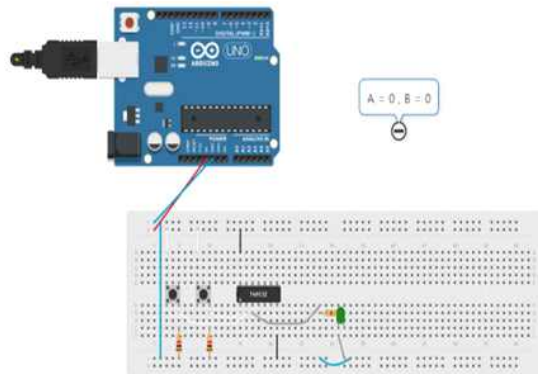
4. $A = 1, B = 1$



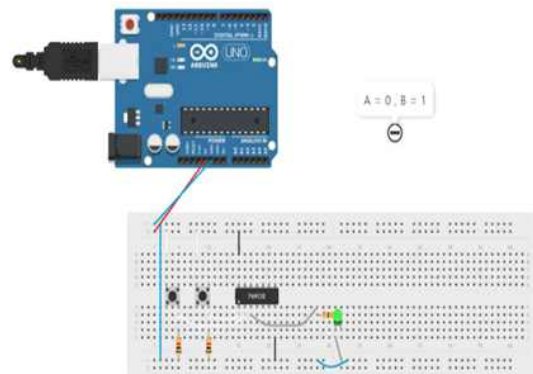
▶ OR 및 NOR 게이트

- OR 게이트

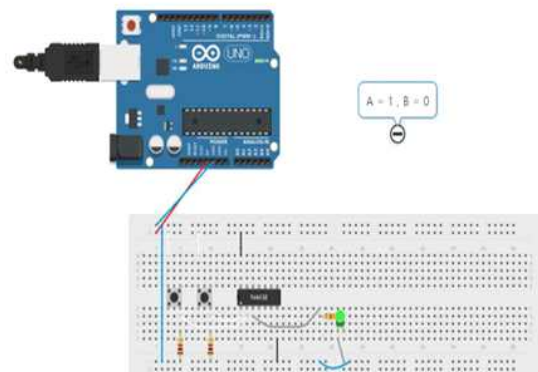
1. $A = 0, B = 0$



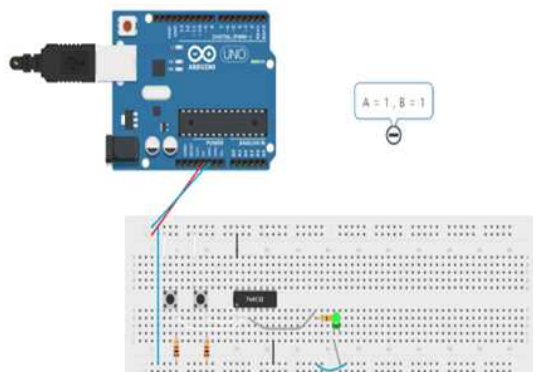
2. $A = 0, B = 1$



3. $A = 1, B = 0$

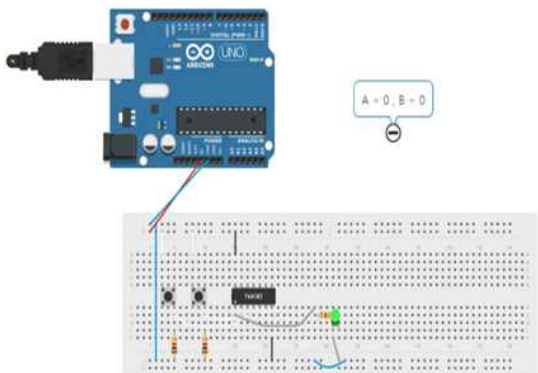


4. $A = 1, B = 1$

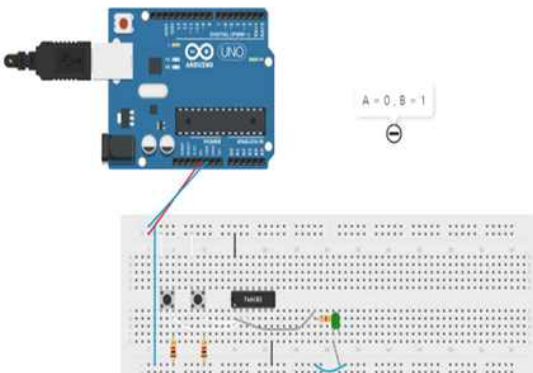


- NOR 게이트

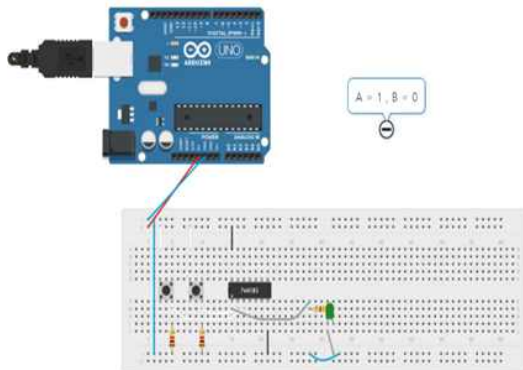
1. $A = 0, B = 0$



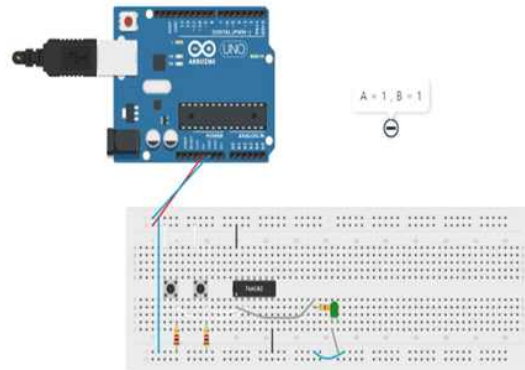
2. $A = 0, B = 1$



3. $A = 1, B = 0$

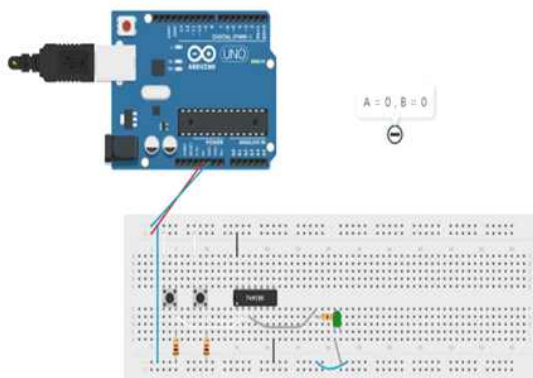


4. $A = 1, B = 1$

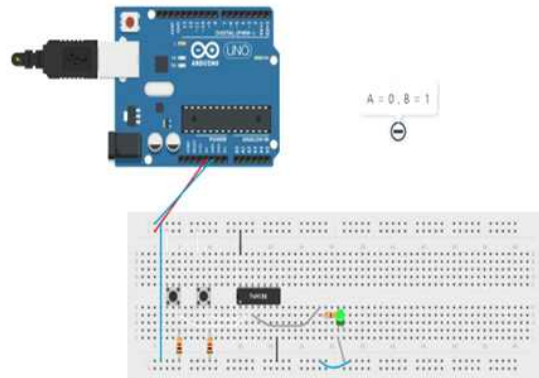


► XOR 게이트

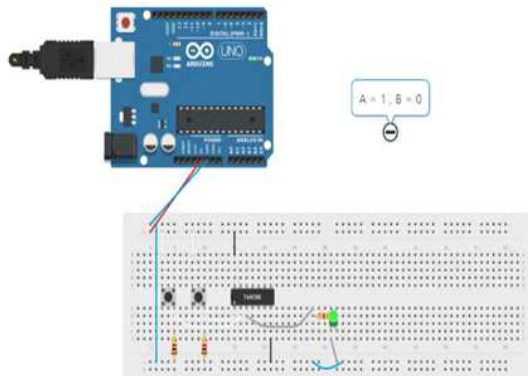
1. $A = 0, B = 0$



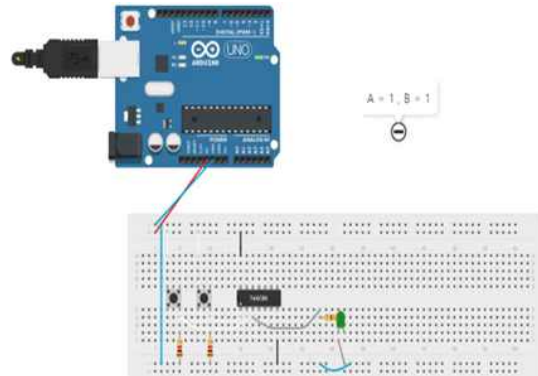
2. $A = 0, B = 1$



3. $A = 1, B = 0$



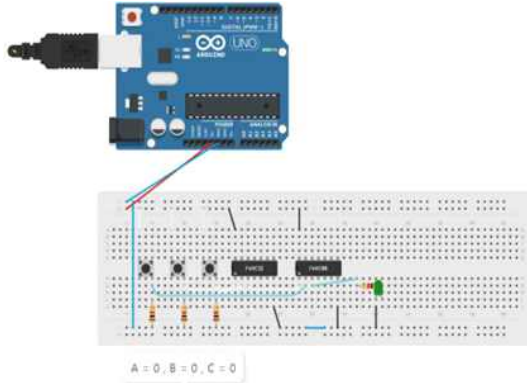
4. $A = 1, B = 1$



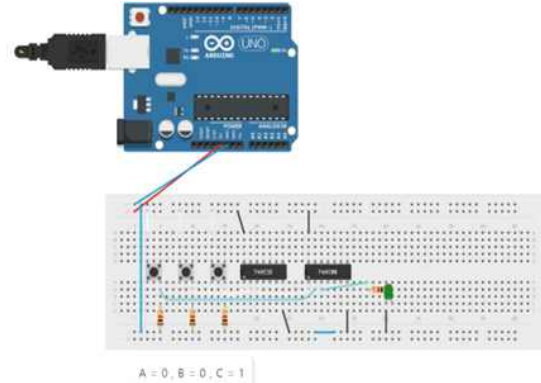
▶ 분배 법칙

- $A \cdot (B + C)$

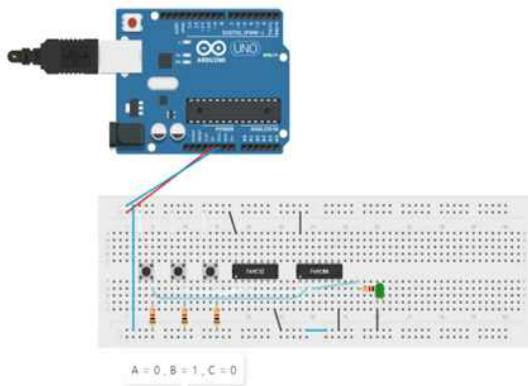
1. $A = 0, B = 0, C = 0$



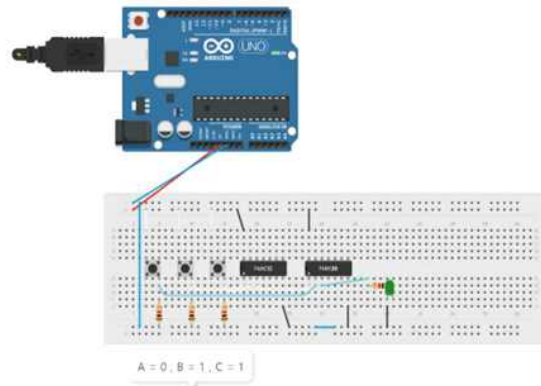
2. $A = 0, B = 0, C = 1$



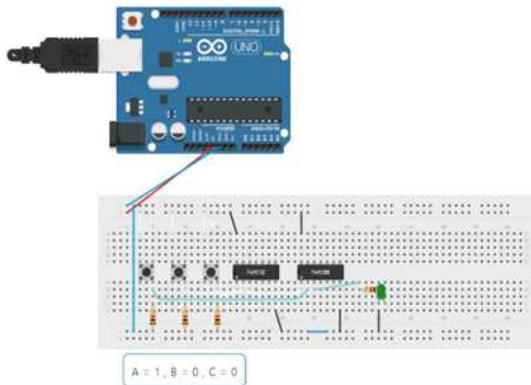
3. $A = 0, B = 1, C = 0$



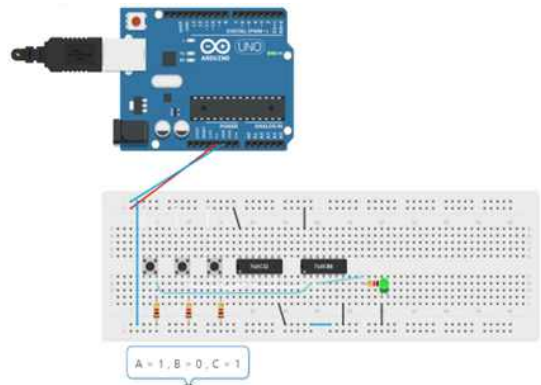
4. $A = 0, B = 1, C = 1$



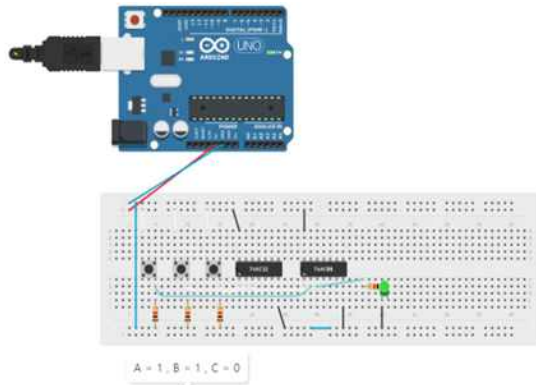
5. $A = 1, B = 0, C = 0$



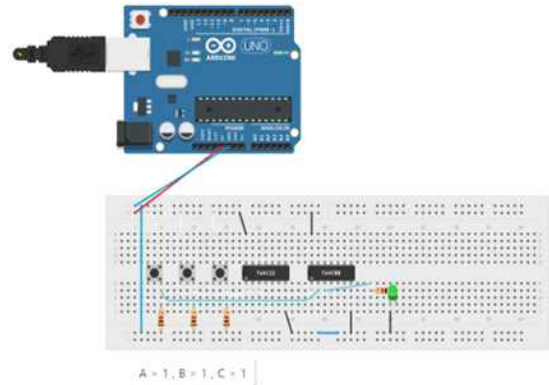
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

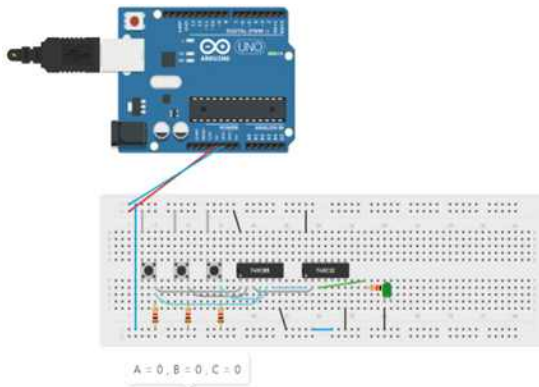


8. $A = 1, B = 1, C = 1$

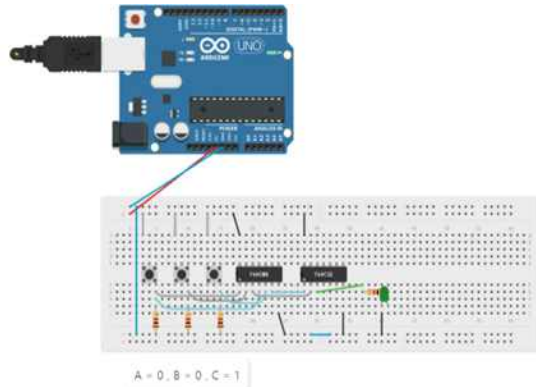


- $A \cdot B + A \cdot C$

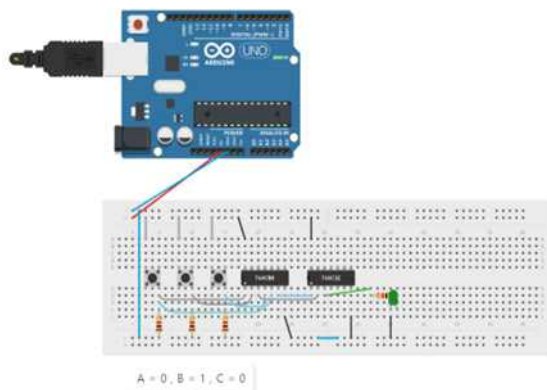
1. $A = 0, B = 0, C = 0$



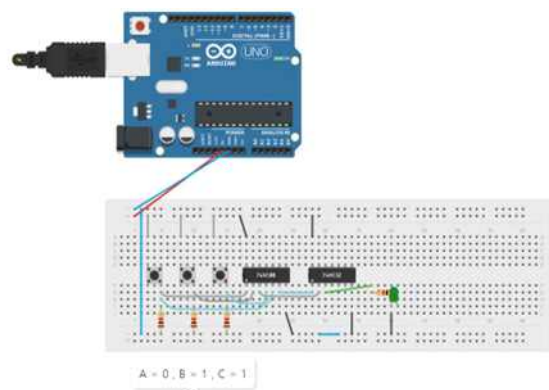
2. $A = 0, B = 0, C = 1$



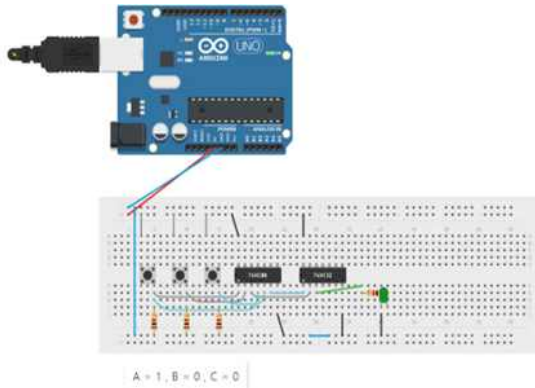
3. $A = 0, B = 1, C = 0$



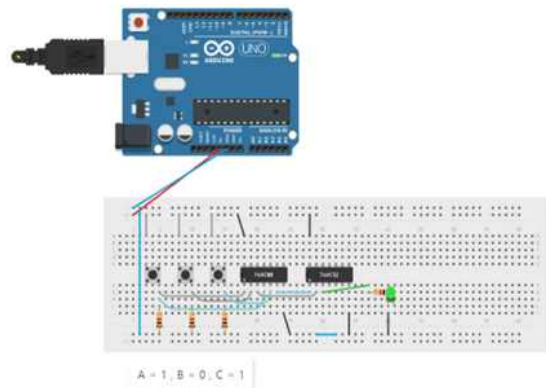
4. $A = 0, B = 1, C = 1$



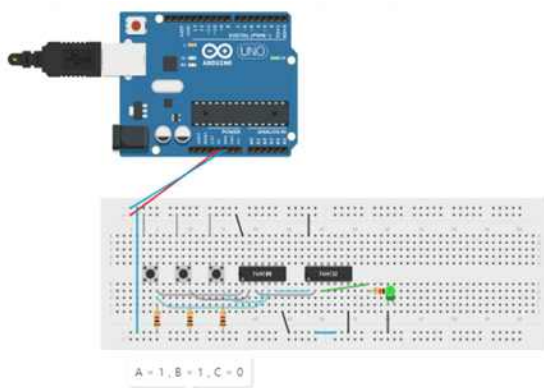
5. $A = 1, B = 0, C = 0$



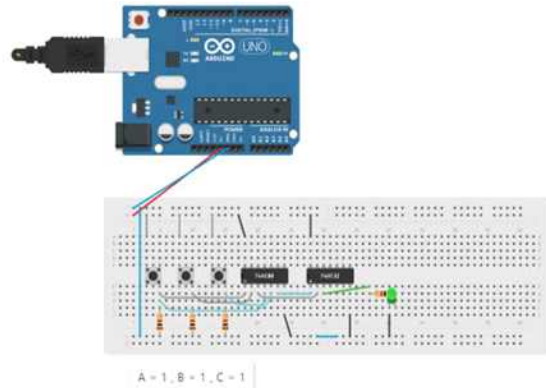
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

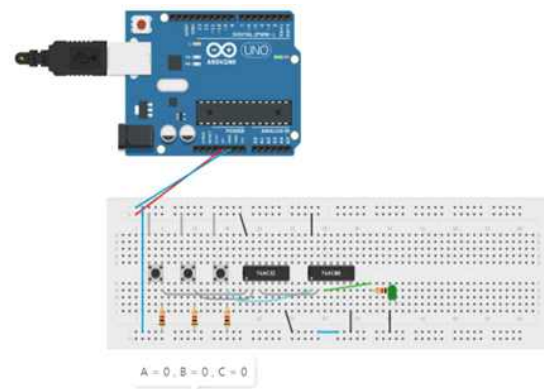


8. $A = 1, B = 1, C = 1$

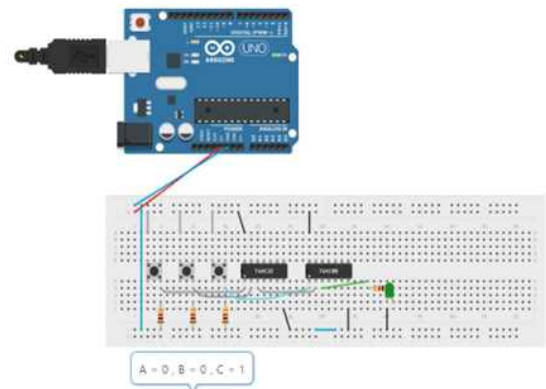


- $(A+B) \cdot C$

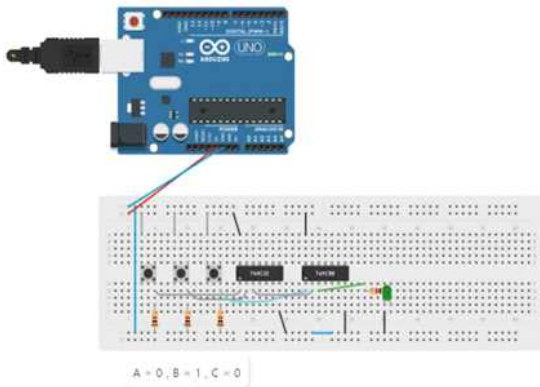
1. $A = 0, B = 0, C = 0$



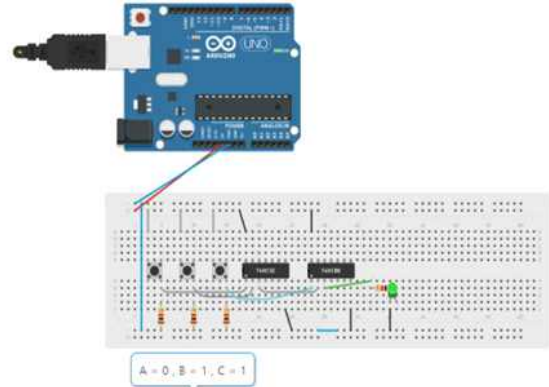
2. $A = 0, B = 0, C = 1$



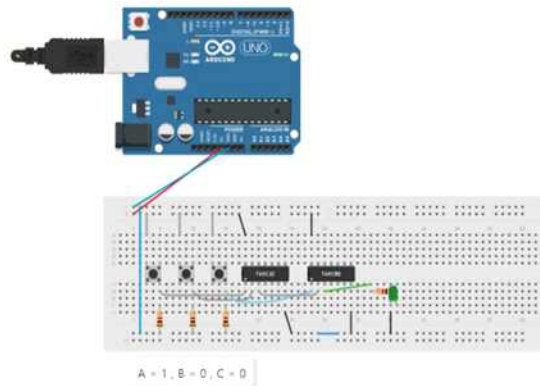
3. $A = 0, B = 1, C = 0$



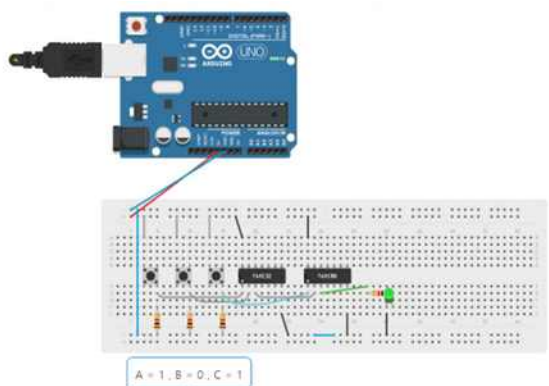
4. $A = 0, B = 1, C = 1$



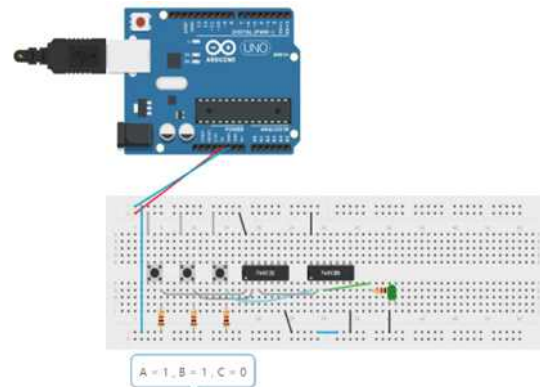
5. $A = 1, B = 0, C = 0$



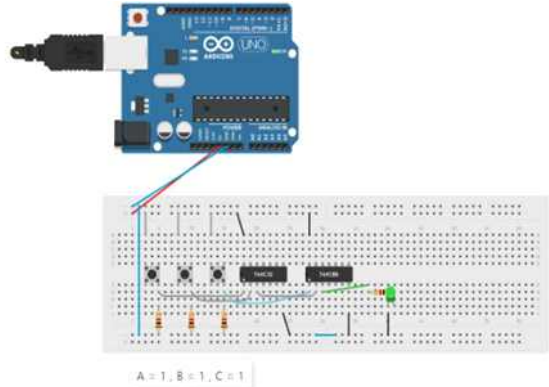
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

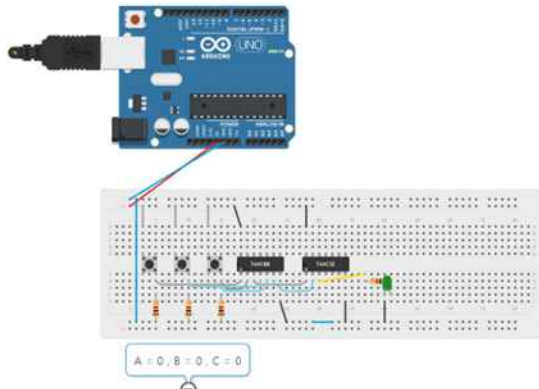


8. $A = 1, B = 1, C = 1$

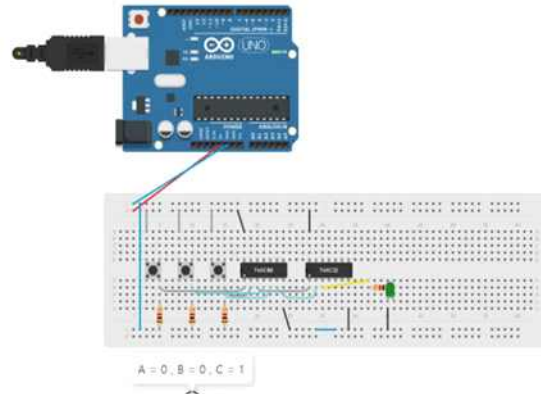


- $A \cdot C + B \cdot C$

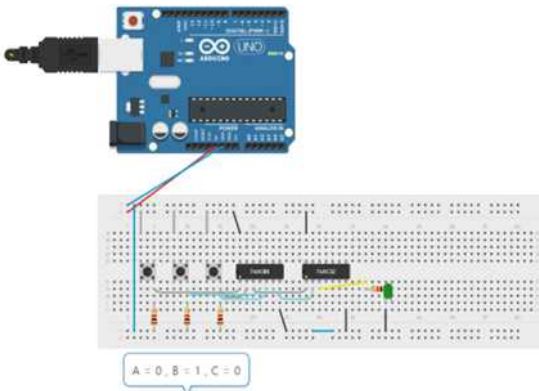
1. $A = 0, B = 0, C = 0$



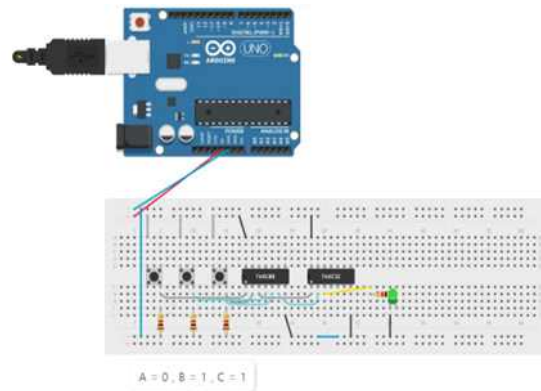
2. $A = 0, B = 0, C = 1$



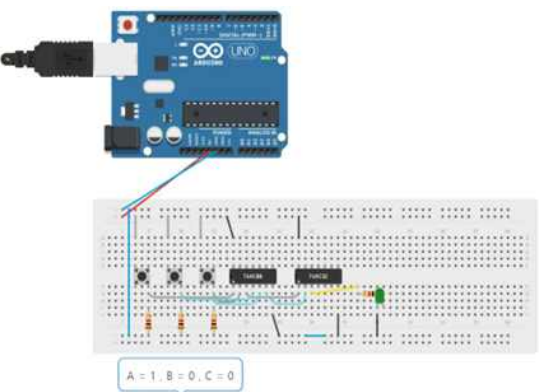
3. $A = 0, B = 1, C = 0$



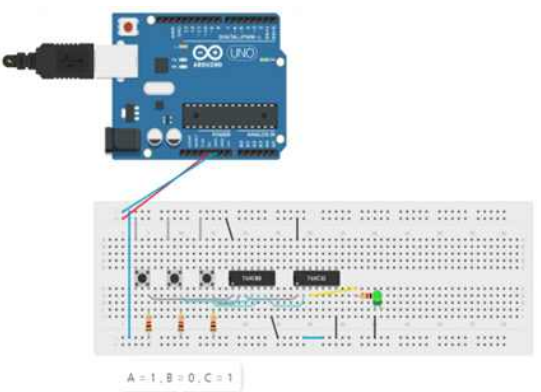
4. $A = 0, B = 1, C = 1$



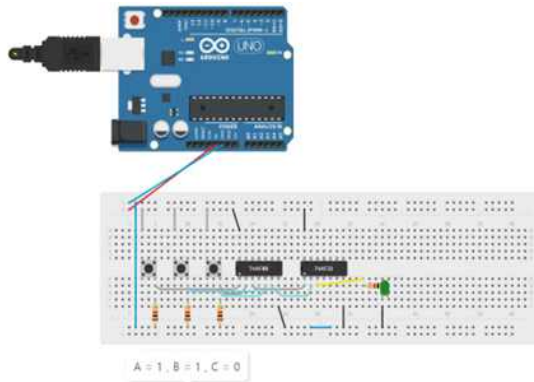
5. $A = 1, B = 0, C = 0$



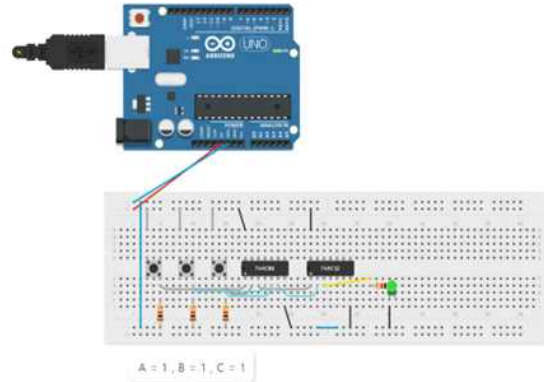
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$



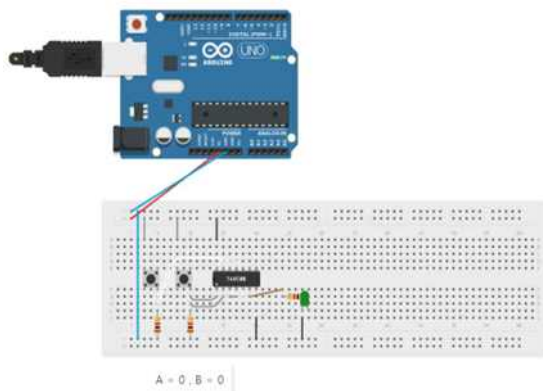
8. $A = 1, B = 1, C = 1$



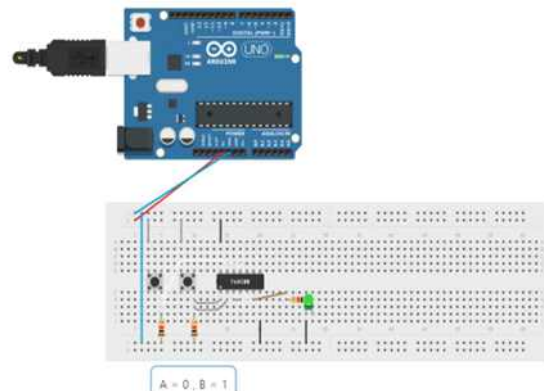
▶ NAND 게이트를 이용한 OR 및 NOR 회로

-NAND 게이트를 이용한 OR

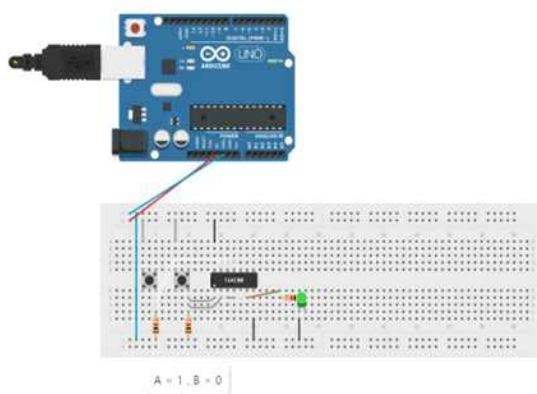
1. $A = 0, B = 0$



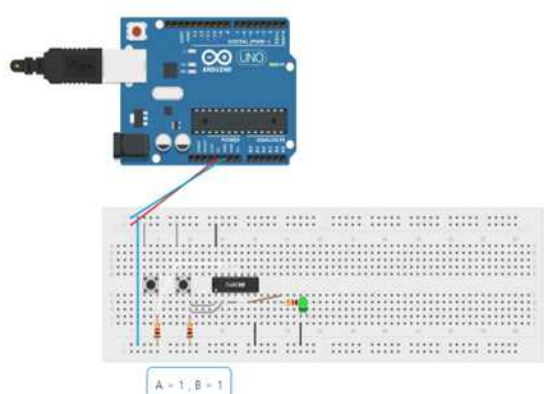
2. $A = 0, B = 1$



3. $A = 1, B = 0$

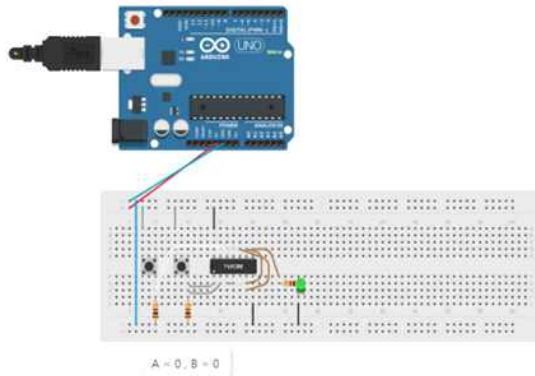


4. $A = 1, B = 1$

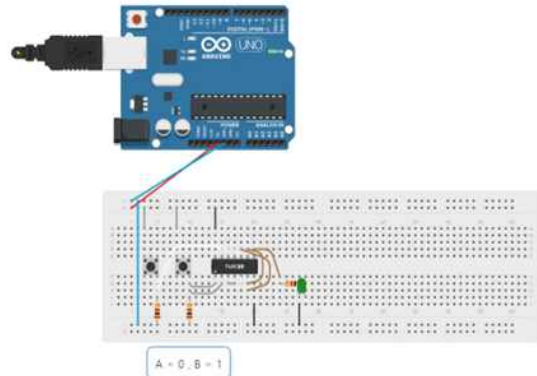


-NAND 게이트를 이용한 NOR

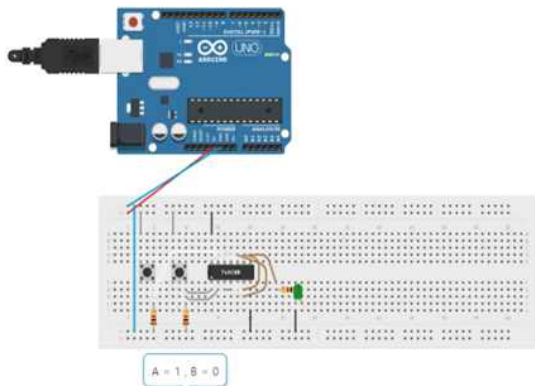
1. $A = 0, B = 0$



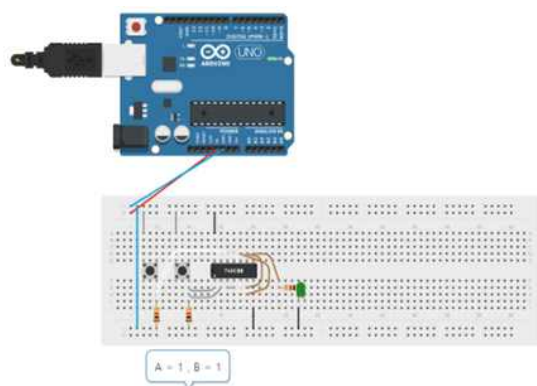
2. $A = 0, B = 1$



3. $A = 1, B = 0$

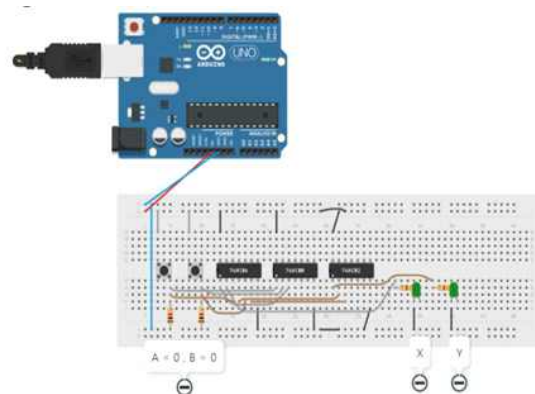


4. $A = 1, B = 1$

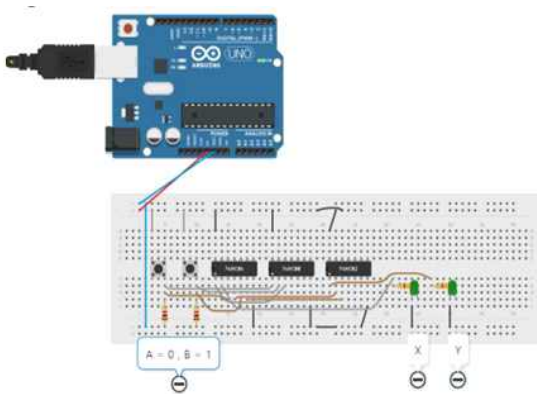


► NOT, NAND, NOR 게이트를 이용한 AND 회로

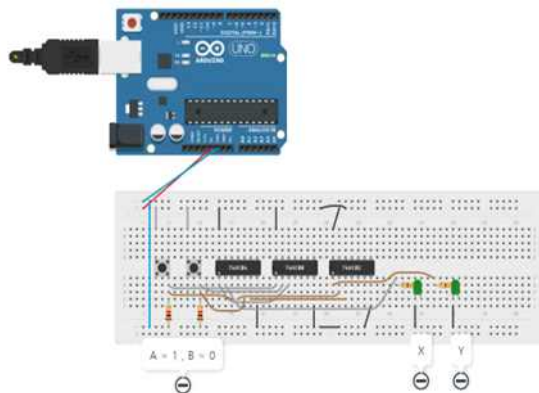
1. $A = 0, B = 0$



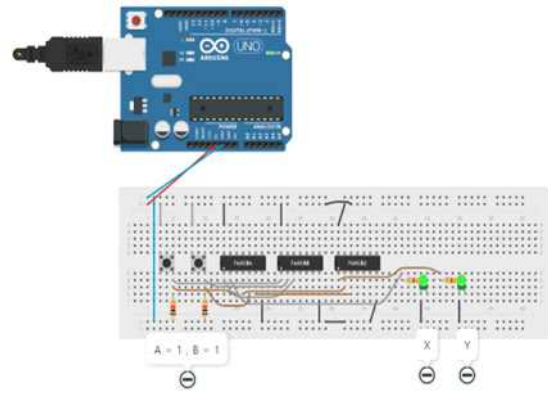
2. $A = 0, B = 1$



3. $A = 1, B = 0$

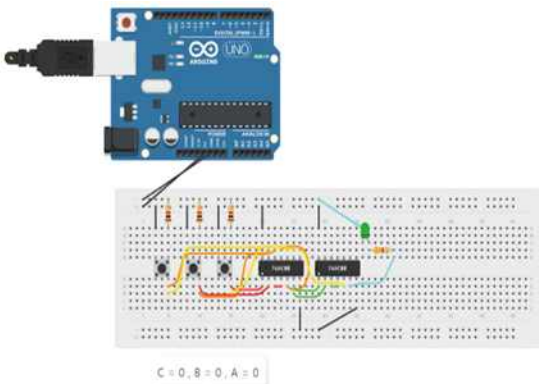


4. $A = 1, B = 1$

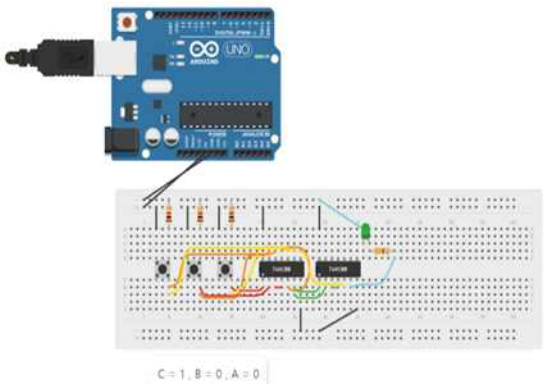


▶ 카르노 맵을 간략화하고, 간략화된 논리식을 NAND 게이트로 구성

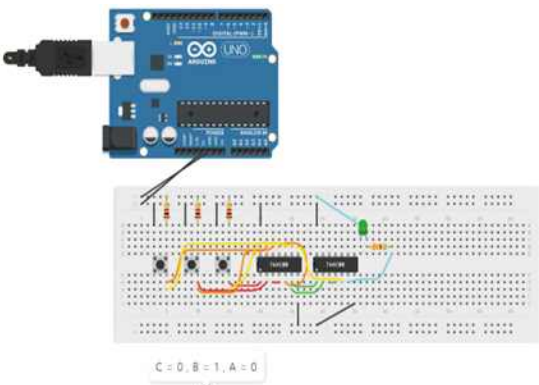
1. $A = 0, B = 0, C = 0$



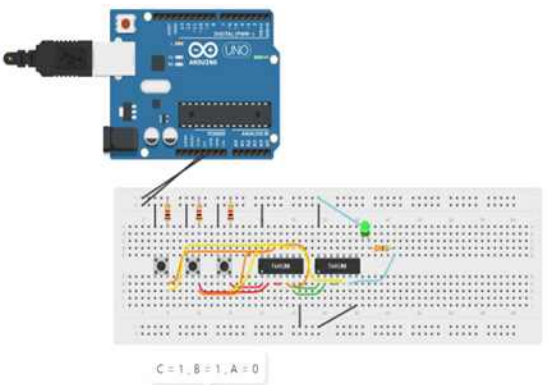
2. $A = 0, B = 0, C = 1$



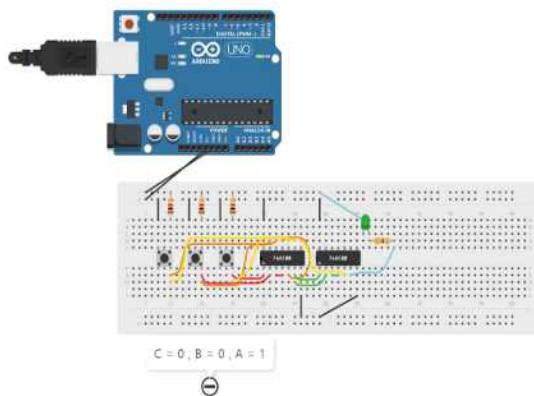
3. $A = 0, B = 1, C = 0$



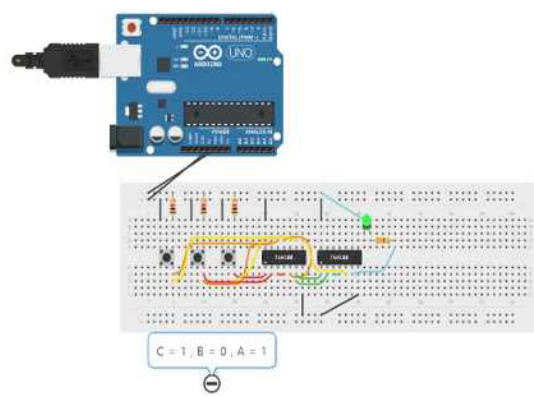
4. $A = 0, B = 1, C = 1$



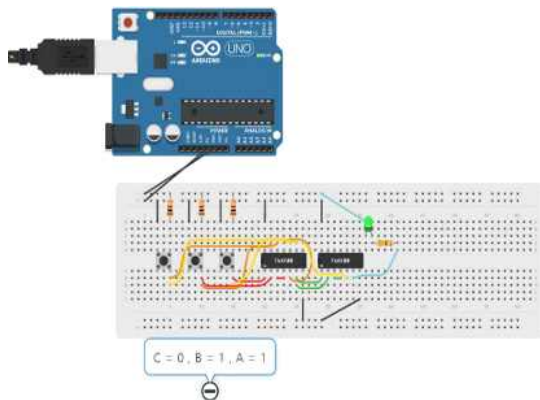
5. $A = 1, B = 0, C = 0$



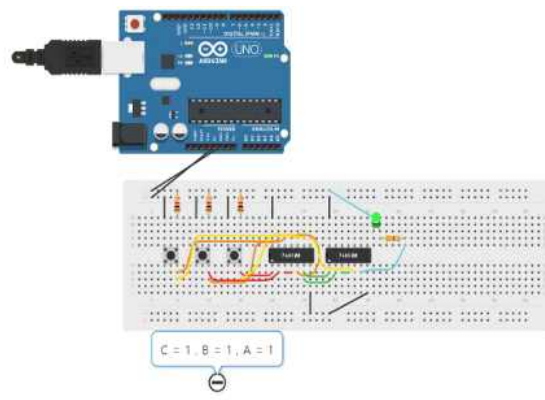
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

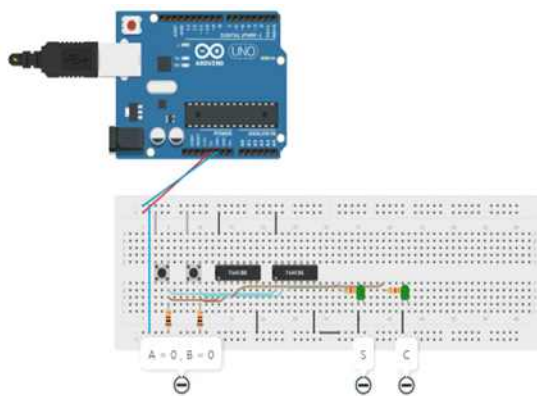


8. $A = 1, B = 1, C = 1$

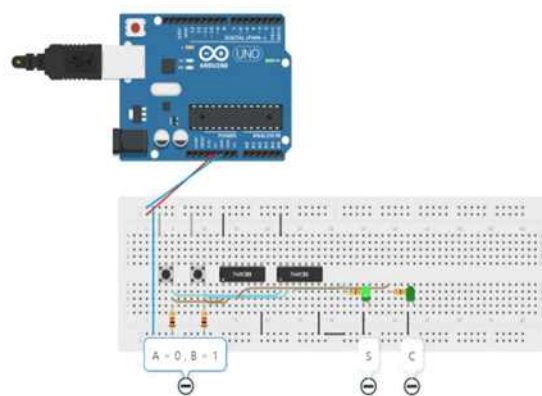


▶ 반가산기 회로

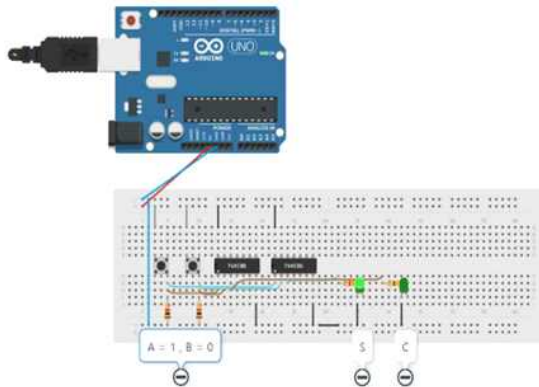
1. $A = 0, B = 0$



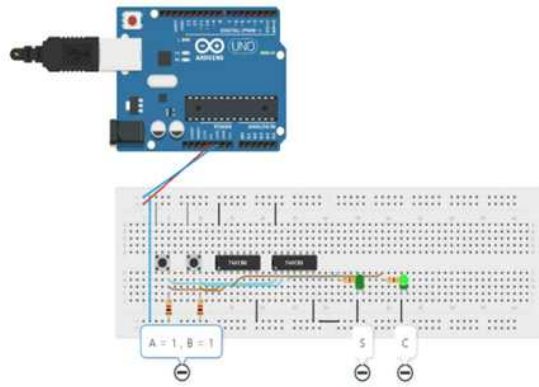
2. $A = 0, B = 1$



3. $A = 1, B = 0$

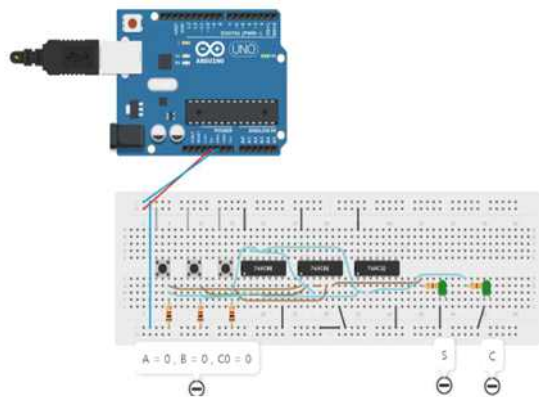


4. $A = 1, B = 1$

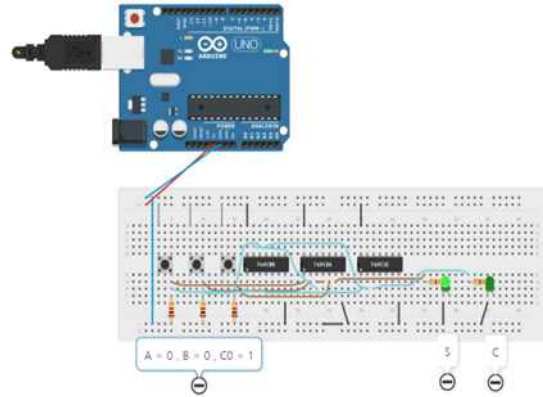


▶ 전가산기 회로

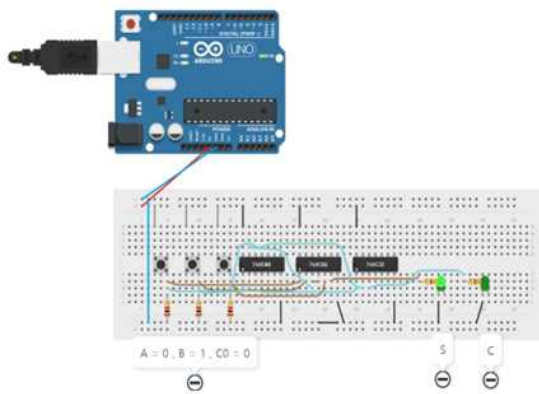
1. $A = 0, B = 0, C_0 = 0$



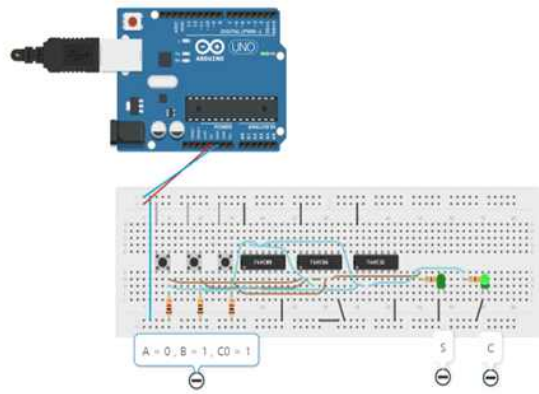
2. $A = 0, B = 0, C_0 = 1$



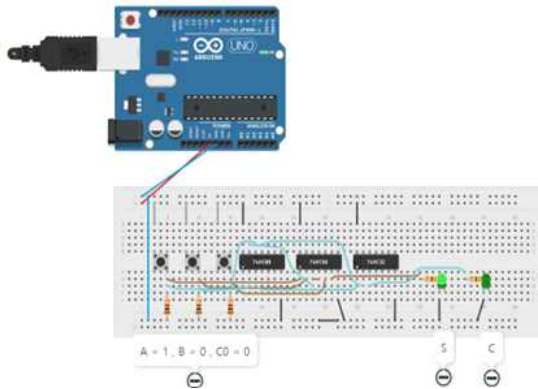
3. $A = 0, B = 1, C_0 = 0$



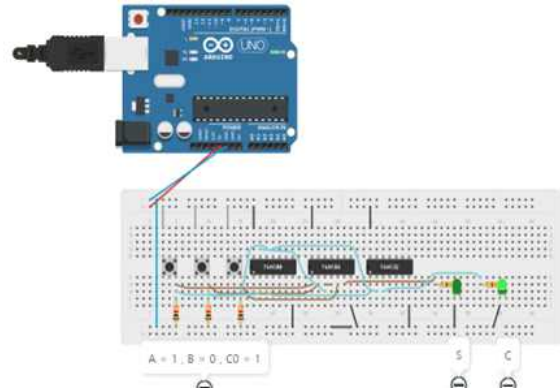
4. $A = 0, B = 1, C_0 = 1$



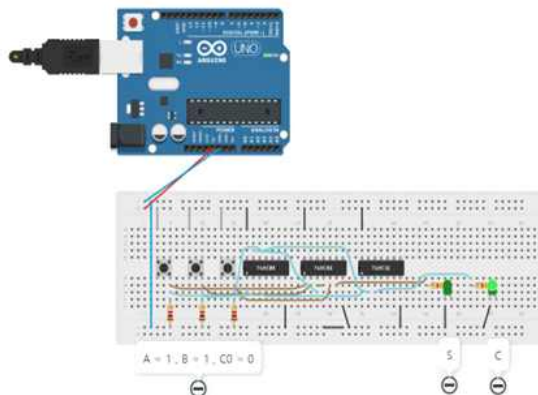
5. $A = 1, B = 0, C0 = 0$



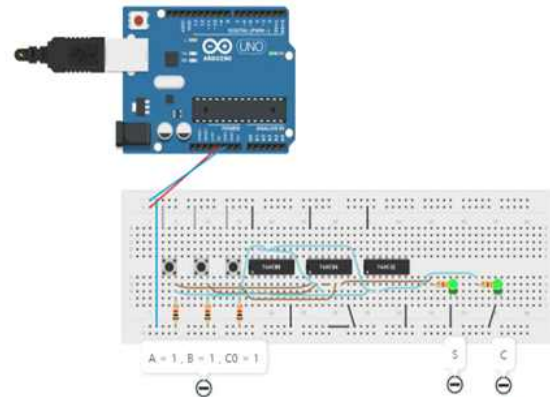
6. $A = 1, B = 0, C0 = 1$



7. $A = 1, B = 1, C0 = 0$

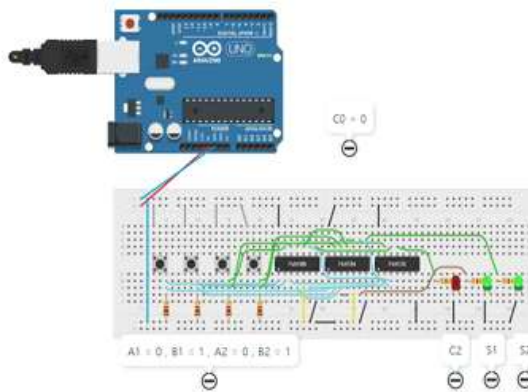


8. $A = 1, B = 1, C0 = 1$

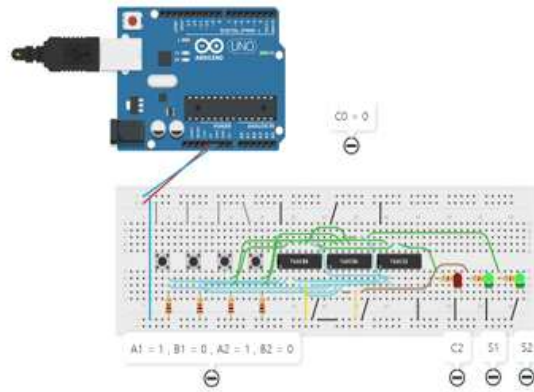


▶ 2비트 2진 가산기

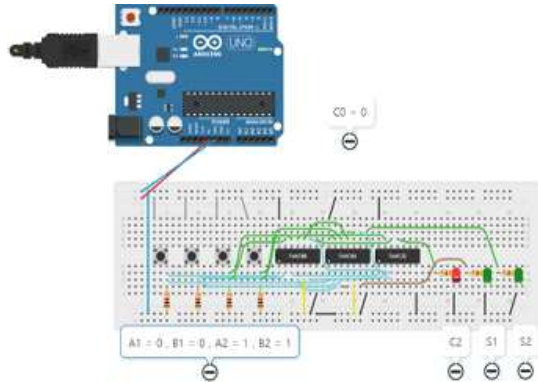
1. $A1=0, B1=1, A2=0, B2=1, C0=0$



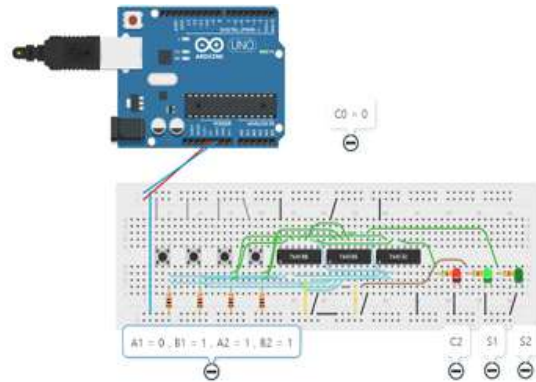
1. $A1=1, B1=0, A2=1, B2=0, C0=0$



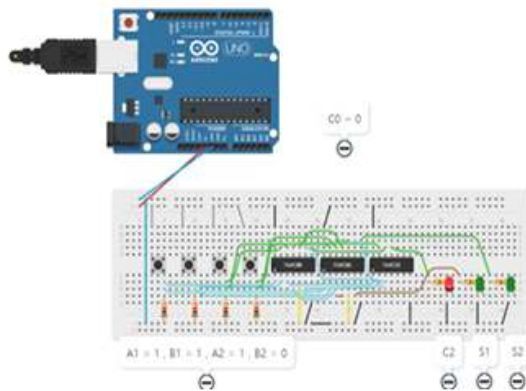
3. $A1=0, B1=0, A2=1, B2=1, C0=0$



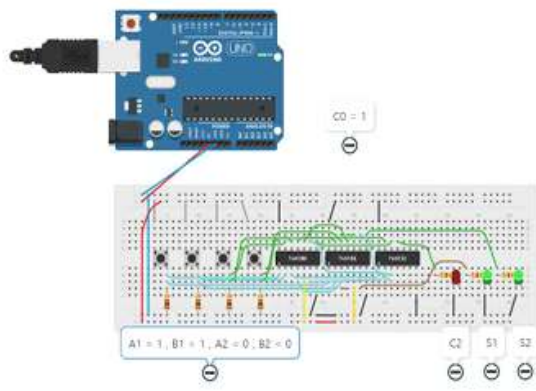
4. $A1=0, B1=1, A2=1, B2=1, C0=0$



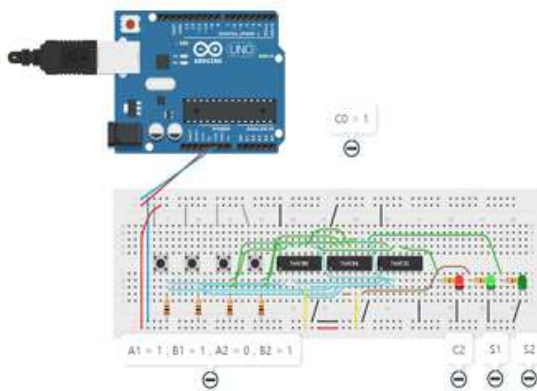
5. $A1=1, B1=1, A2=1, B2=0, C0=0$



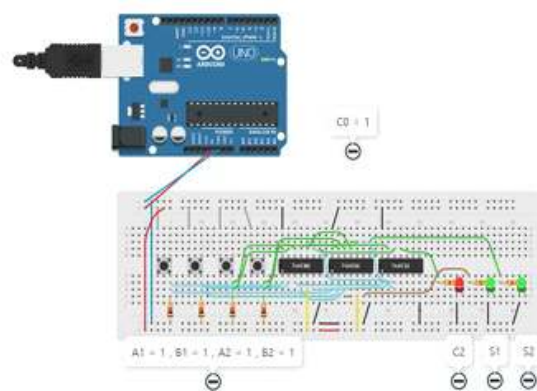
6. $A1=1, B1=1, A2=0, B2=0, C0=1$



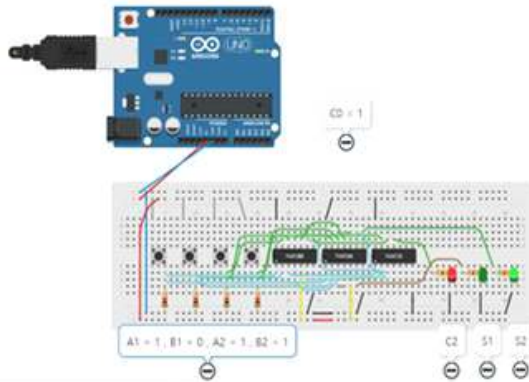
7. $A1=1, B1=1, A2=0, B2=1, C0=1$



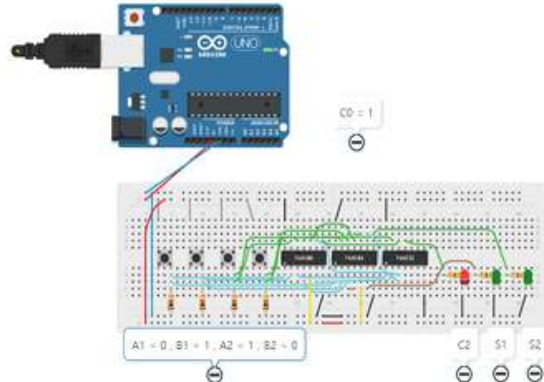
8. $A1=1, B1=1, A2=1, B2=1, C0=1$



9. $A1=1, B1=0, A2=1, B2=1, C0=1$

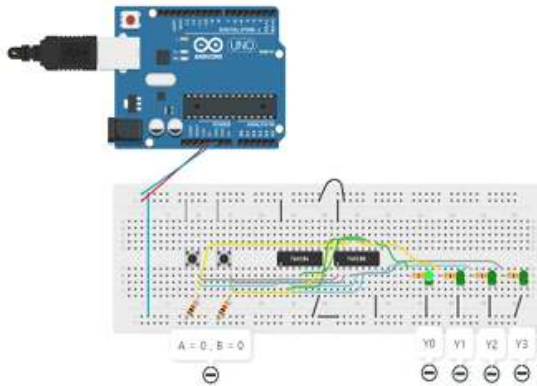


10. $A1=0, B1=1, A2=1, B2=0, C0=1$

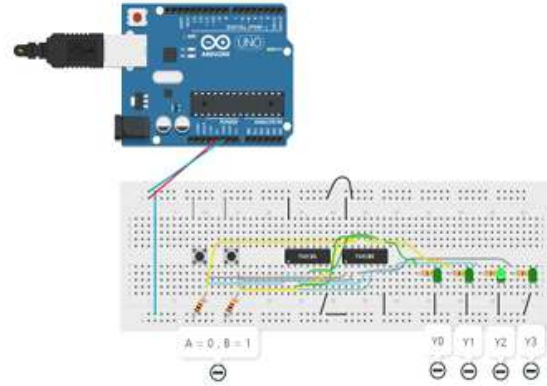


▶ 2x4 디코더 회로

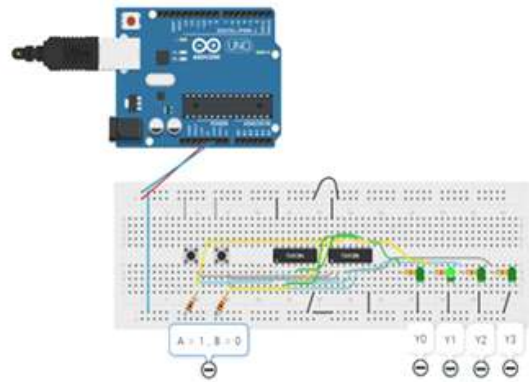
1. $A = 0, B = 0$



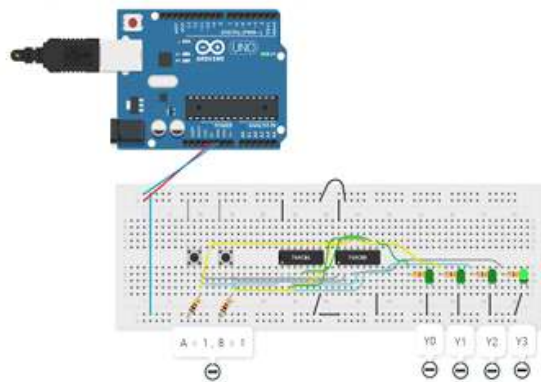
2. $A = 0, B = 1$



3. $A = 1, B = 0$

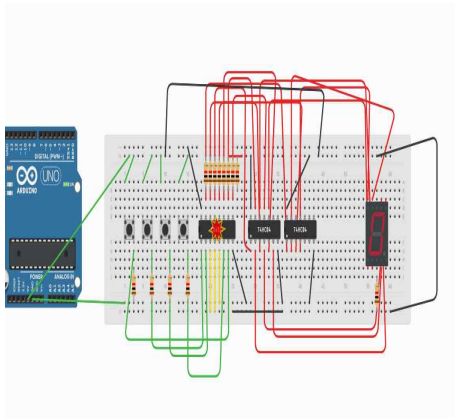


4. $A = 1, B = 1$

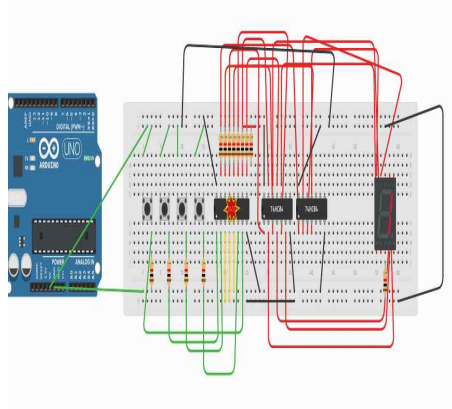


▶ 7세그먼트 회로

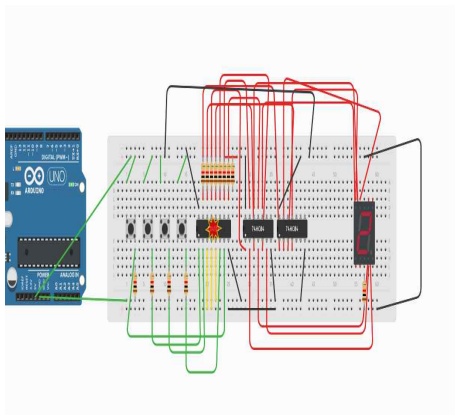
1. A=0, B=0, C=0, D=0



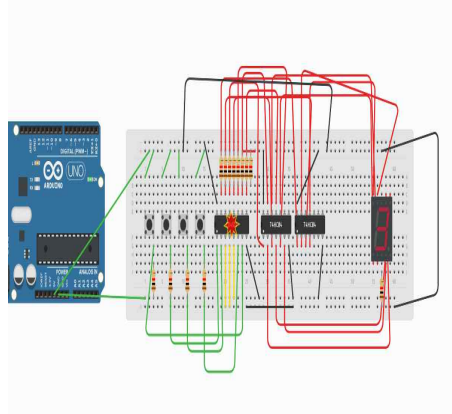
2. A=1, B=0, C=0, D=0



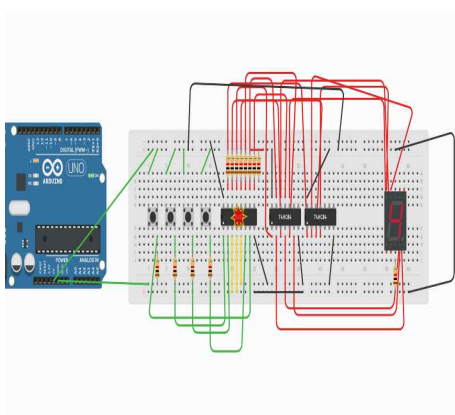
3. A=0, B=1, C=0, D=0



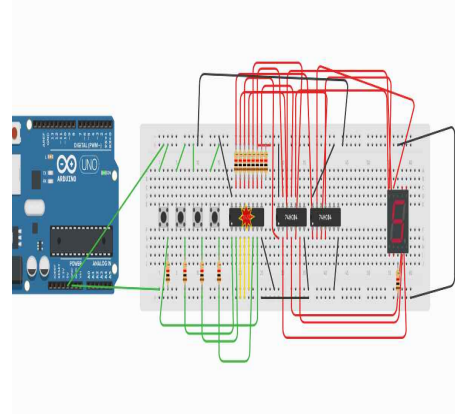
4. A=1, B=1, C=0, D=0



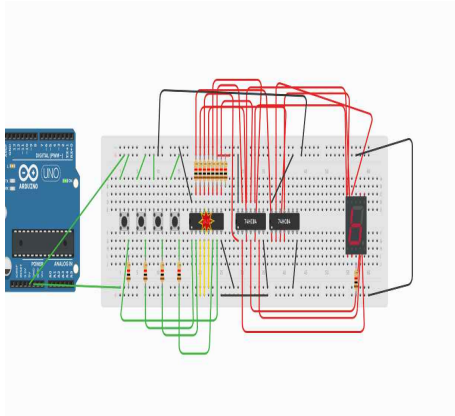
5. A=0, B=0, C=1, D=0



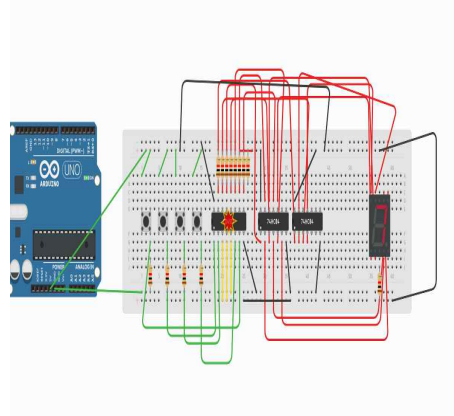
6. A=1, B=0, C=1, D=0



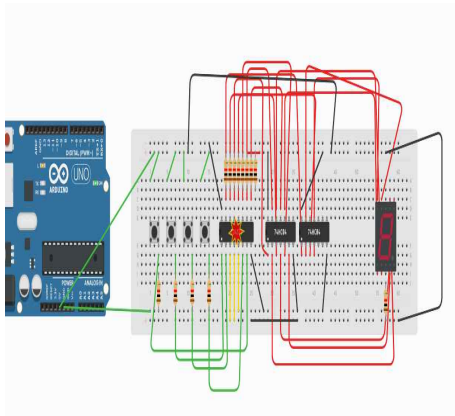
7. A=0, B=1, C=1, D=0



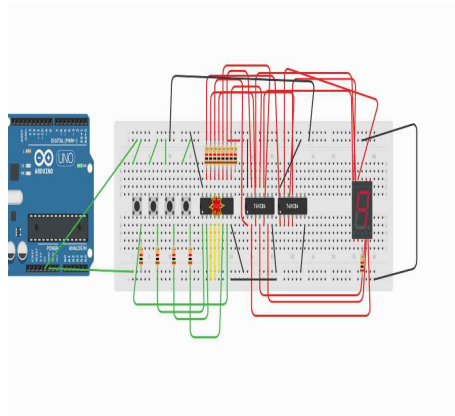
8. A=1, B=1, C=1, D=0



9. A=0, B=0, C=0, D=1



10. A=1, B=0, C=0, D=1

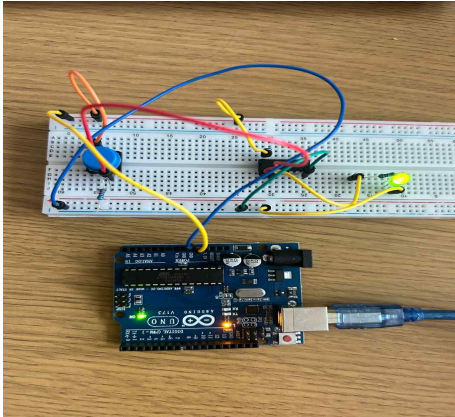


<Arduino Kit를 이용한 실습>

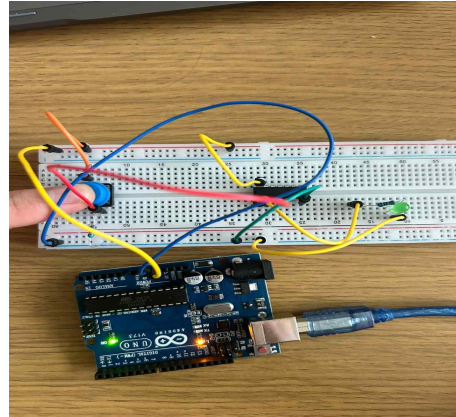
- Arduino kit를 이용하여 게이트를 제작 및 실습

- NOT 게이트

1. $A = 0$



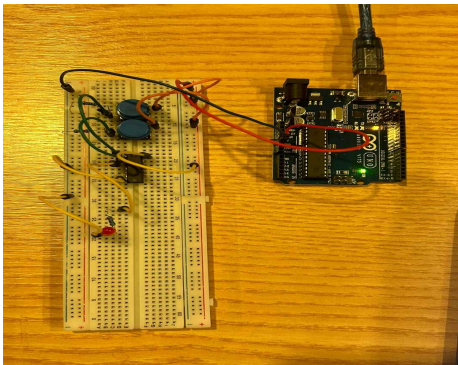
2. $A = 1$



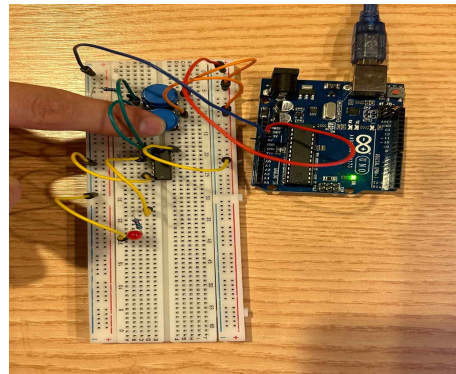
- AND 및 NAND 게이트

-AND 게이트

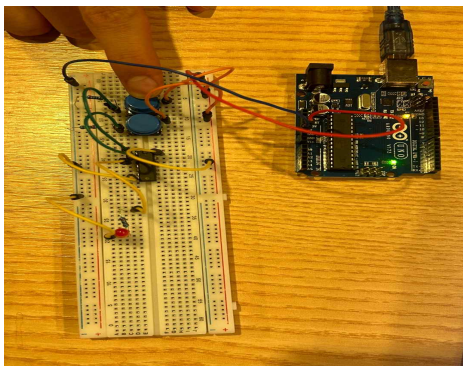
1. $A = 0, B = 0$



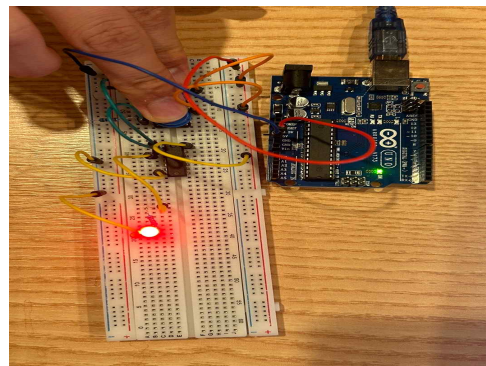
2. $A = 0, B = 1$



3. $A = 1, B = 0$

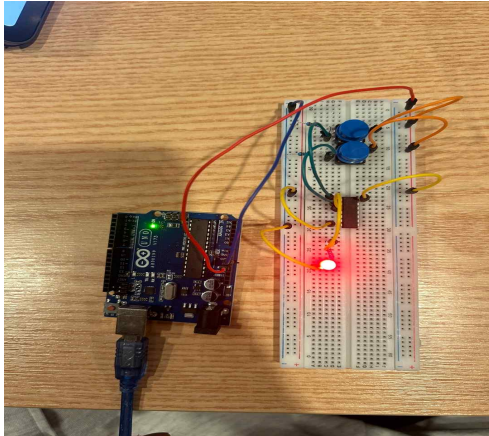


4. $A = 1, B = 1$

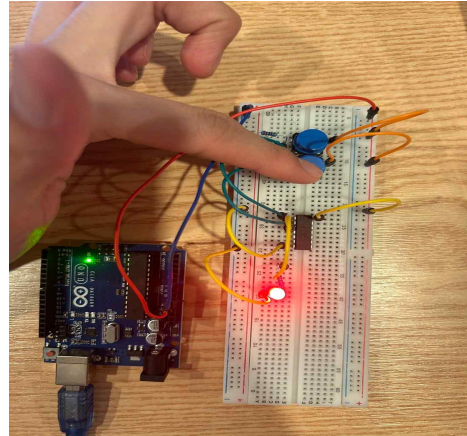


-NAND 게이트

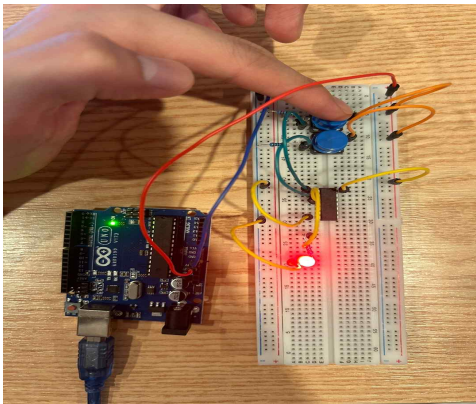
1. $A = 0, B = 0$



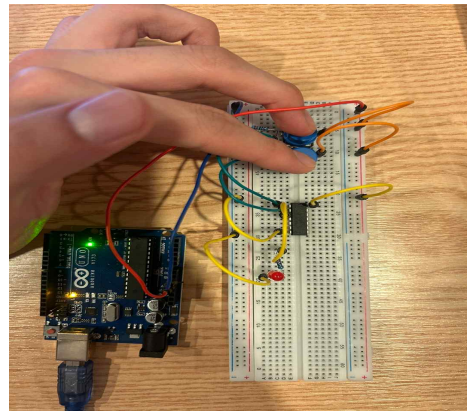
2. $A = 0, B = 1$



3. $A = 1, B = 0$



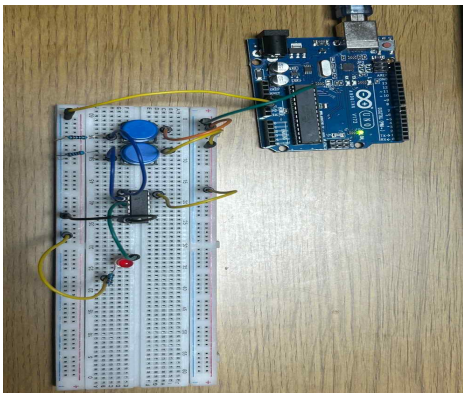
4. $A = 1, B = 1$



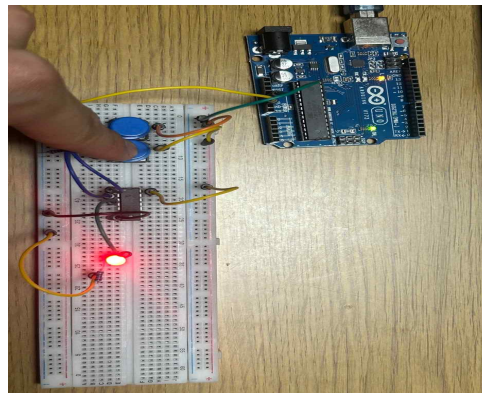
● OR 및 NOR 게이트

-OR 게이트

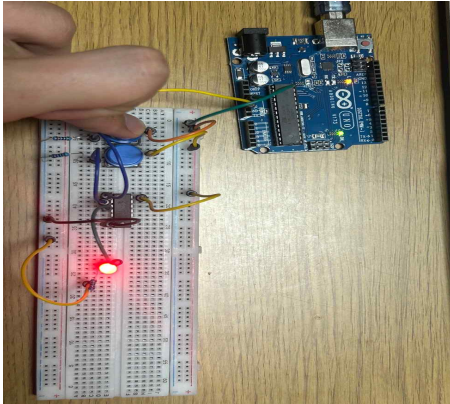
1. $A = 0, B = 0$



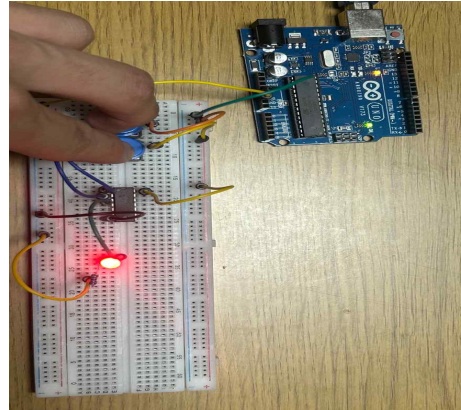
2. $A = 0, B = 1$



3. $A = 1, B = 0$

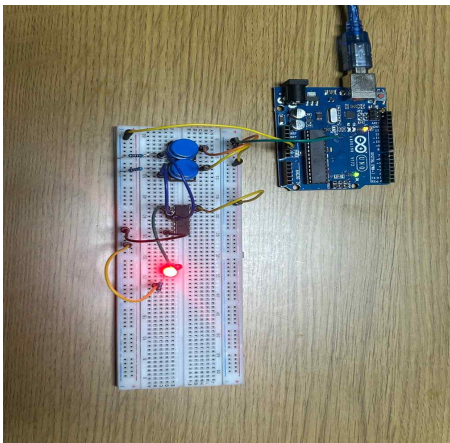


4. $A = 1, B = 1$

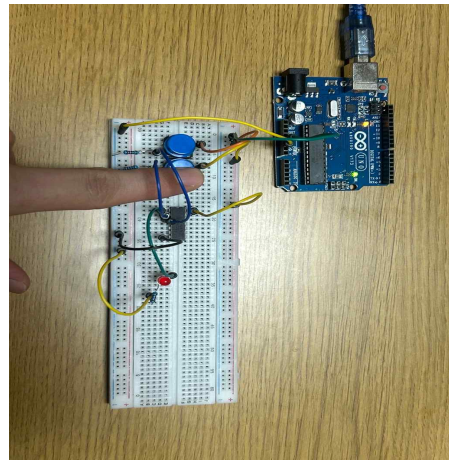


-NOR 게이트

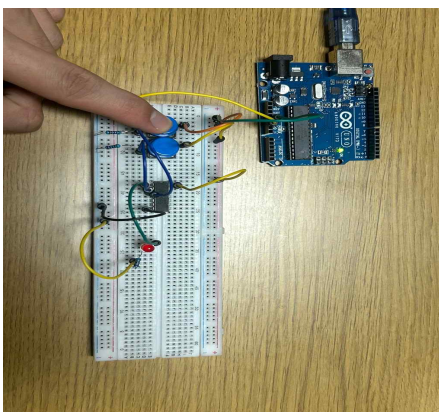
1. $A = 0, B = 0$



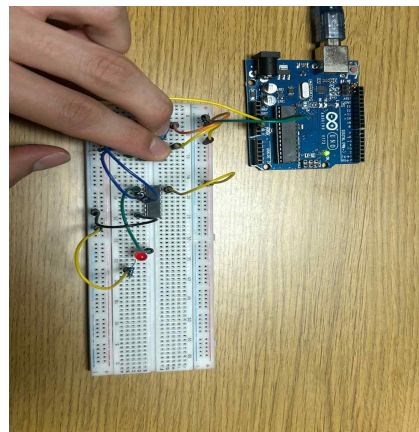
2. $A = 0, B = 1$



3. $A = 1, B = 0$

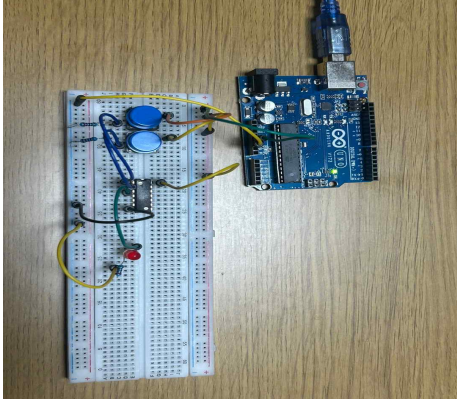


4. $A = 1, B = 1$

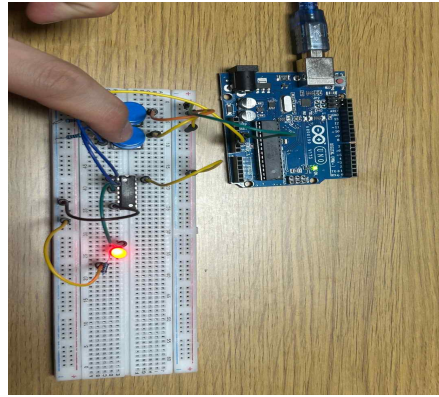


● XOR 게이트

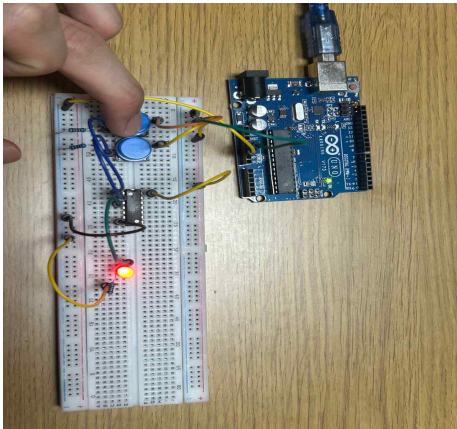
1. $A = 0, B = 0$



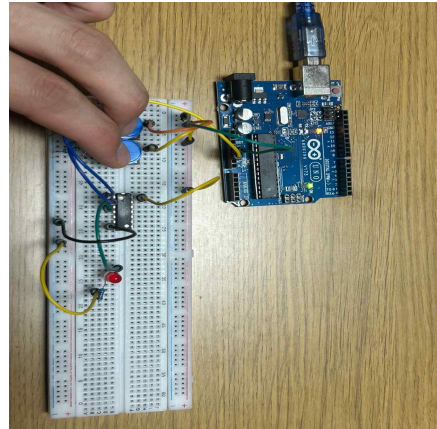
2. $A = 0, B = 1$



3. $A = 1, B = 0$



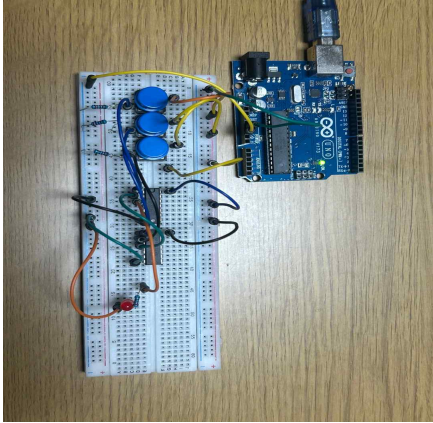
4. $A = 1, B = 1$



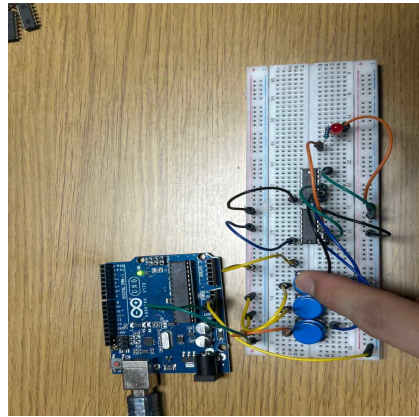
● 분배법칙

- $A \cdot (B + C)$

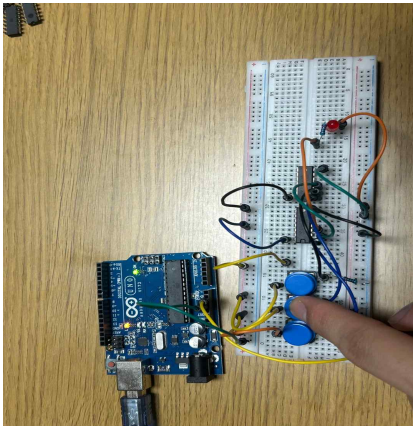
1. $A = 0, B = 0, C = 0$



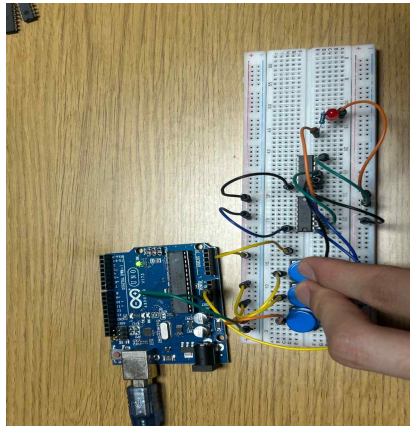
2. $A = 0, B = 0, C = 1$



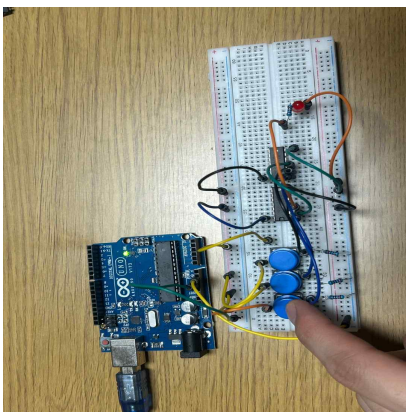
3. $A = 0, B = 1, C = 0$



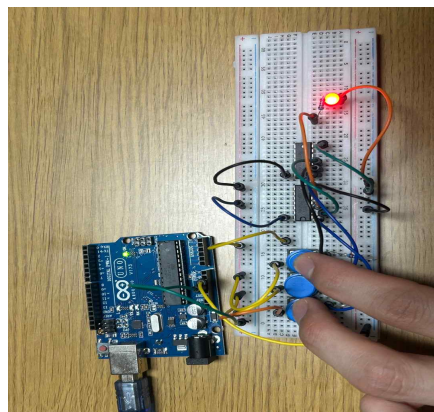
4. $A = 0, B = 1, C = 1$



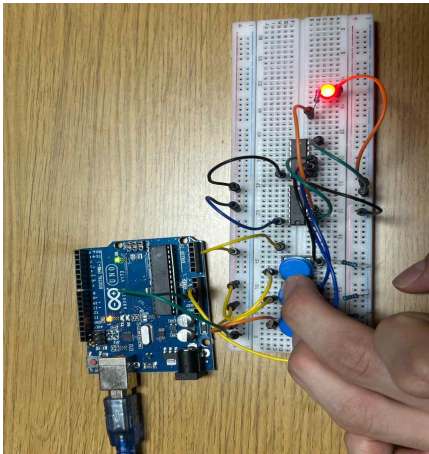
5. $A = 1, B = 0, C = 0$



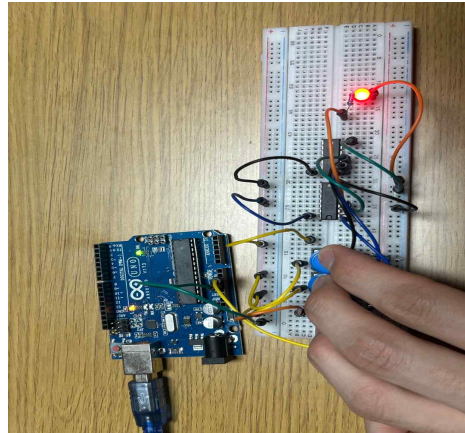
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

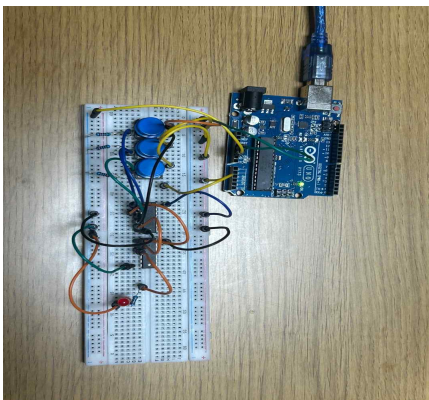


8. $A = 1, B = 1, C = 1$

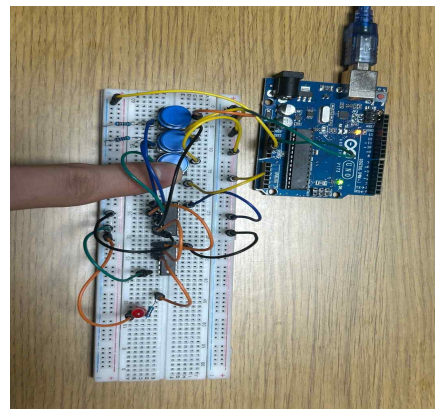


- $A \cdot B + A \cdot C$

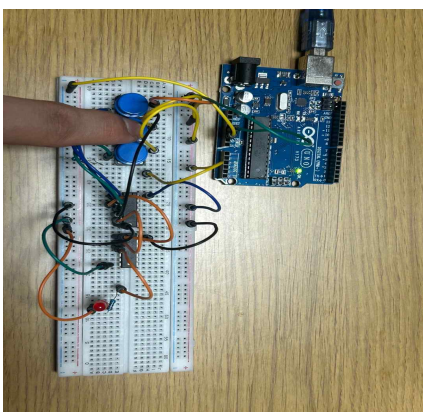
1. $A = 0, B = 0, C = 0$



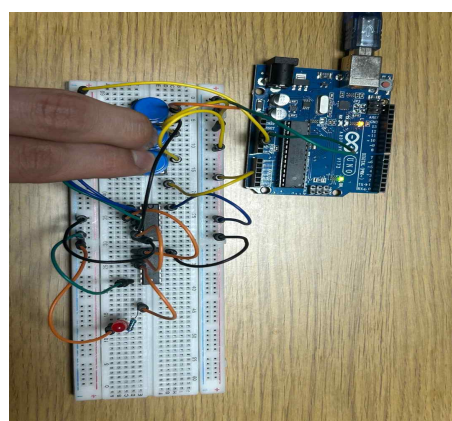
2. $A = 0, B = 0, C = 1$



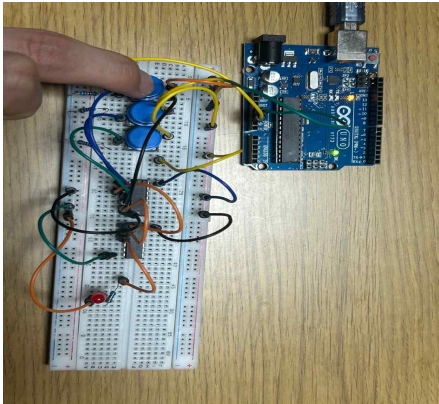
3. $A = 0, B = 1, C = 0$



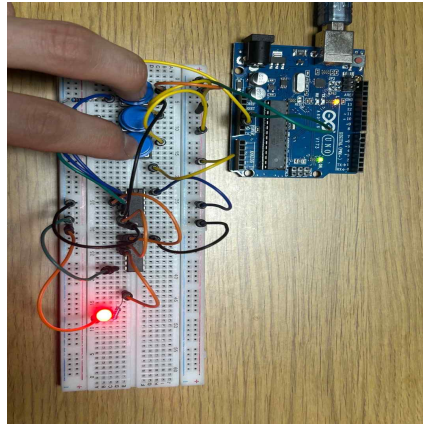
4. $A = 0, B = 1, C = 1$



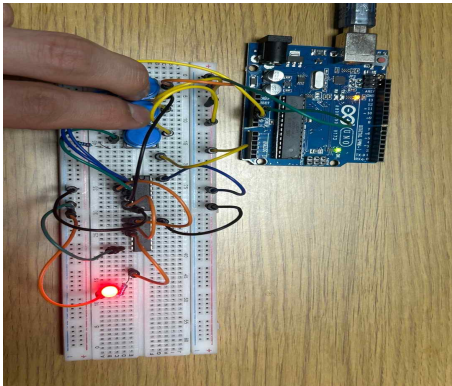
5. $A = 1, B = 0, C = 0$



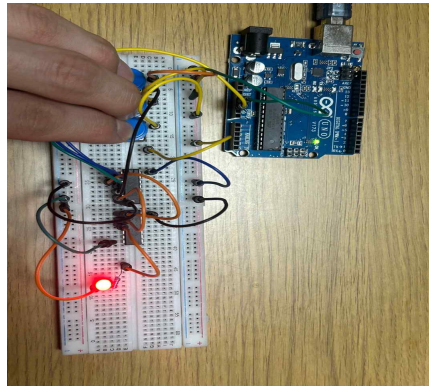
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

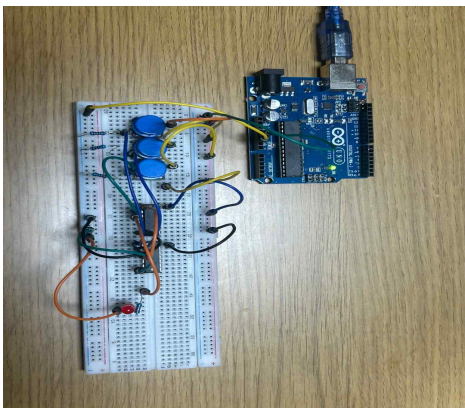


8. $A = 1, B = 1, C = 1$

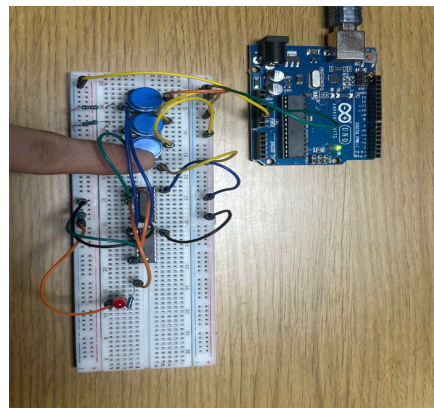


$\neg(A+B) \cdot C$

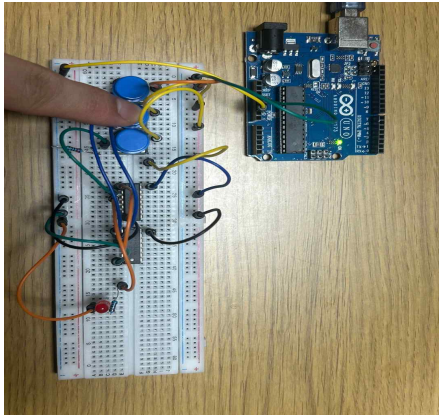
1. $A = 0, B = 0, C = 0$



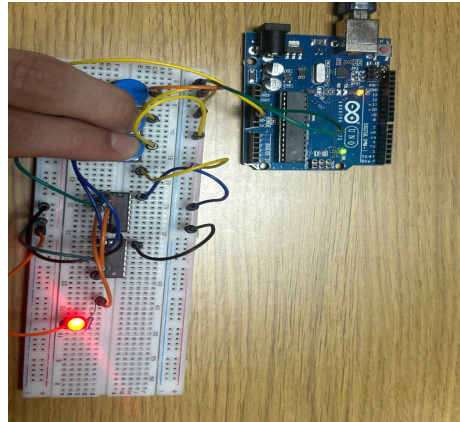
2. $A = 0, B = 0, C = 1$



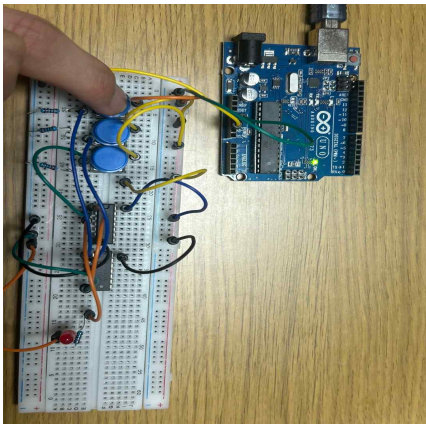
3. $A = 0, B = 1, C = 0$



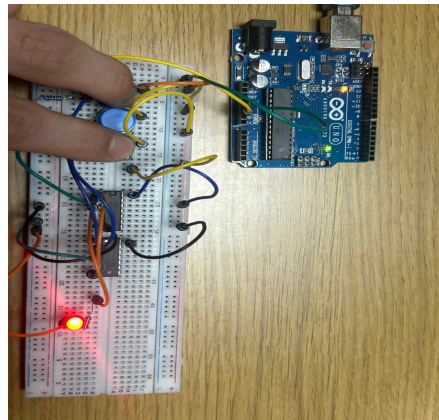
4. $A = 0, B = 1, C = 1$



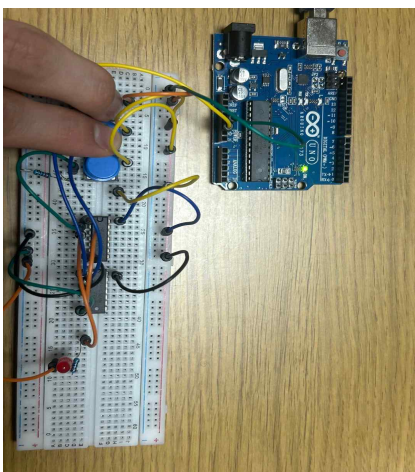
5. $A = 1, B = 0, C = 0$



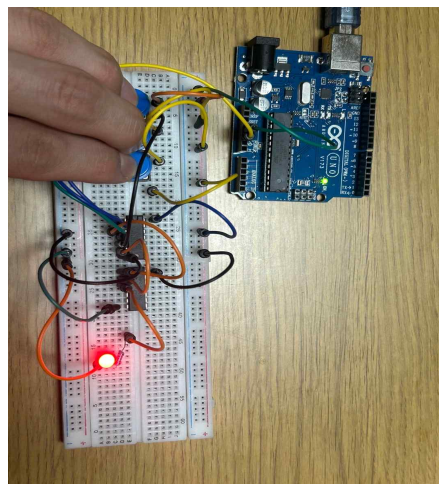
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$

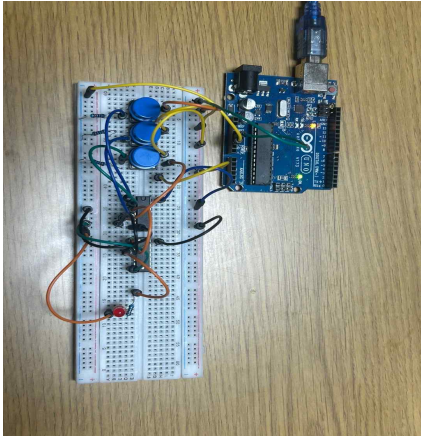


8. $A = 1, B = 1, C = 1$

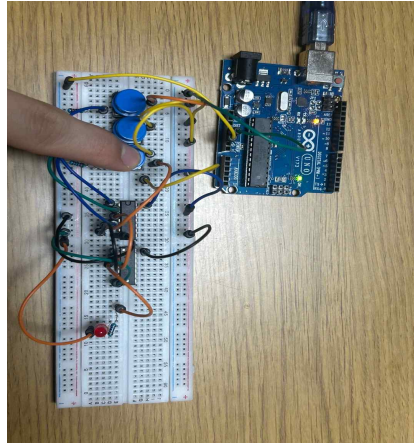


$$- A \cdot C + B \cdot C$$

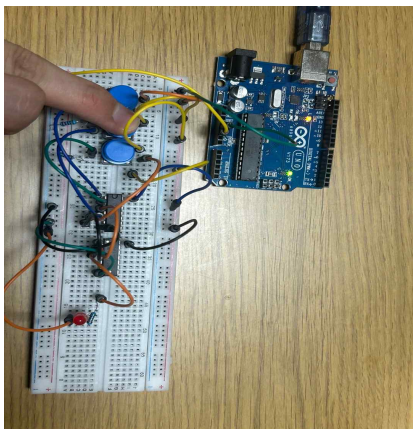
1. $A = 0, B = 0, C = 0$



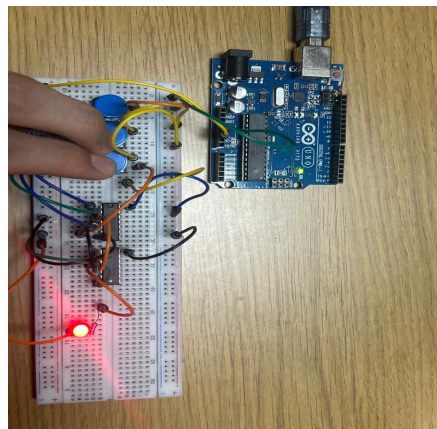
2. $A = 0, B = 0, C = 1$



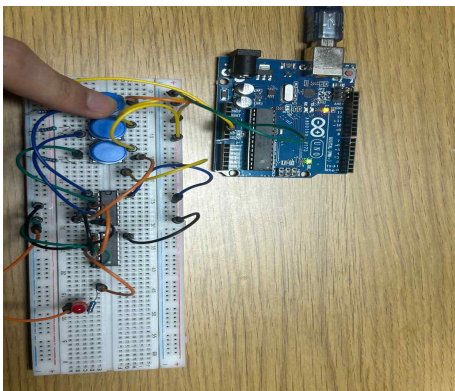
3. $A = 0, B = 1, C = 0$



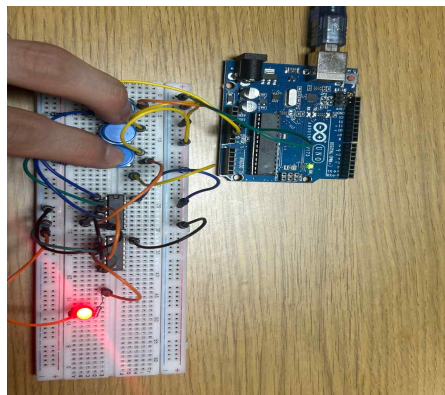
4. $A = 0, B = 1, C = 1$



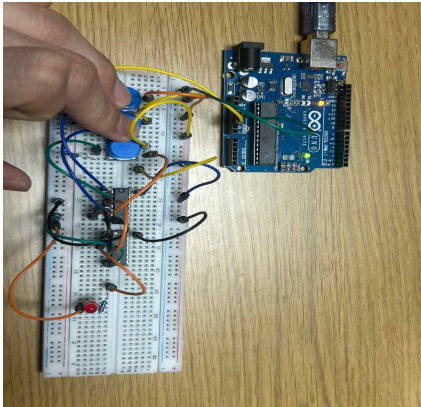
5. $A = 1, B = 0, C = 0$



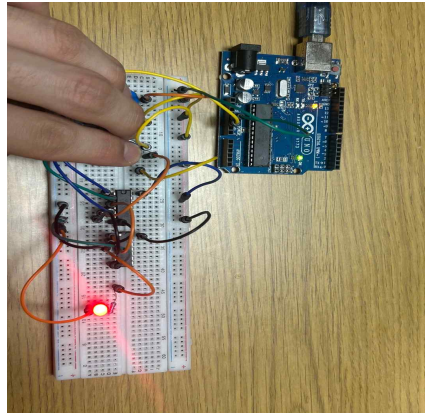
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$



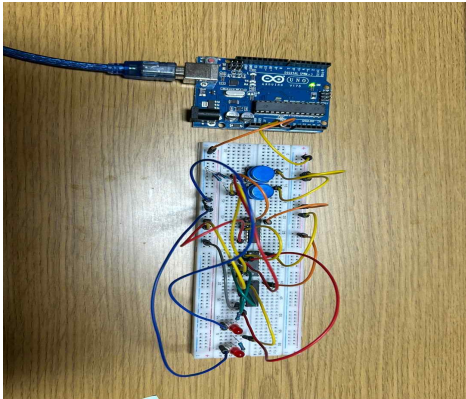
8. $A = 1, B = 1, C = 1$



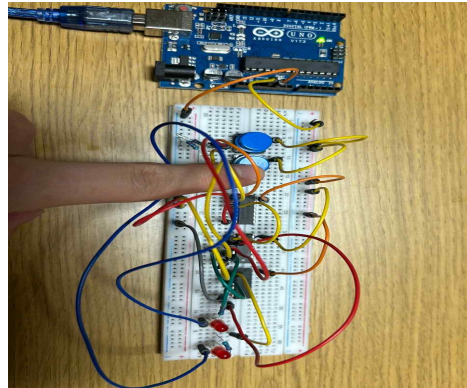
● 분배법칙

-NOT, NOR, NAND를 이용한 AND게이트

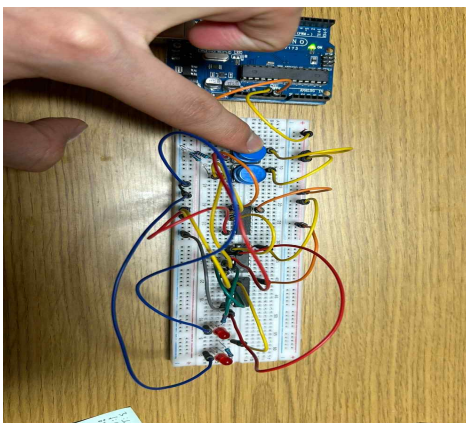
1. $A = 0, B = 0$



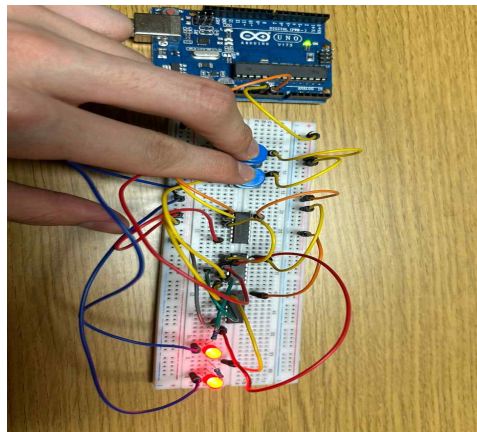
2. $A = 0, B = 1$



3. $A = 1, B = 0$

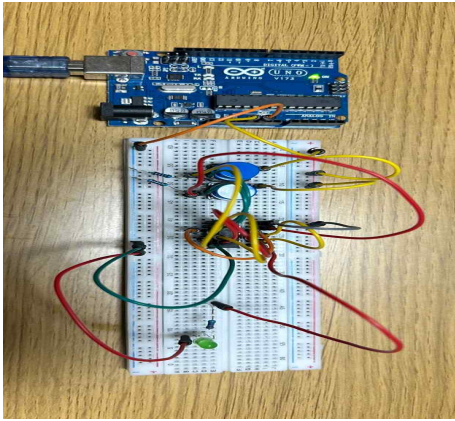


4. $A = 1, B = 1$

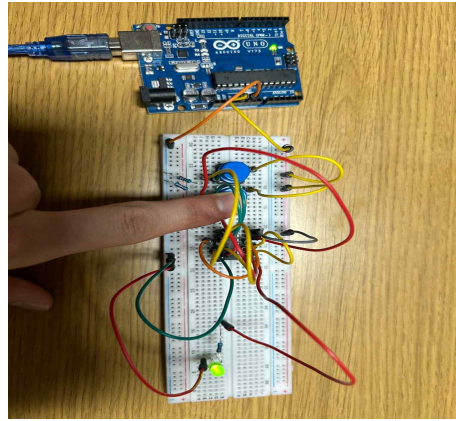


● NOT 게이트를 이용한 XOR게이트

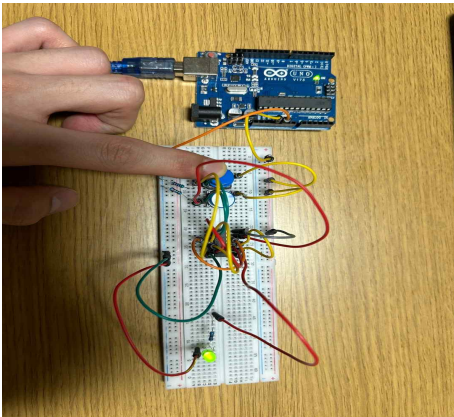
1. $A = 0, B = 0$



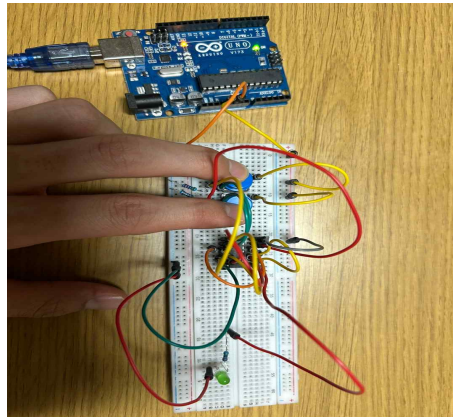
2. $A = 0, B = 1$



3. $A = 1, B = 0$

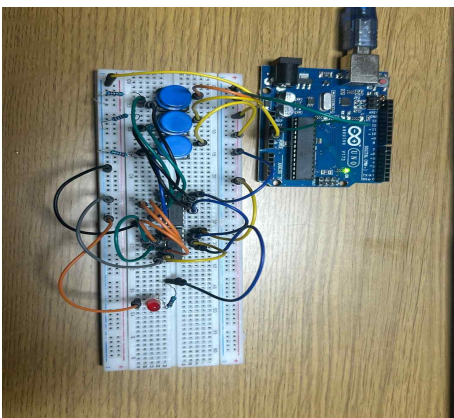


4. $A = 1, B = 1$

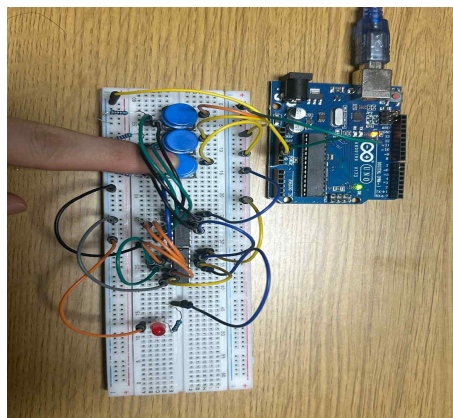


● 카르노 맵을 간략화 하고, 간략화된 논리식을 NAND 게이트로 구성

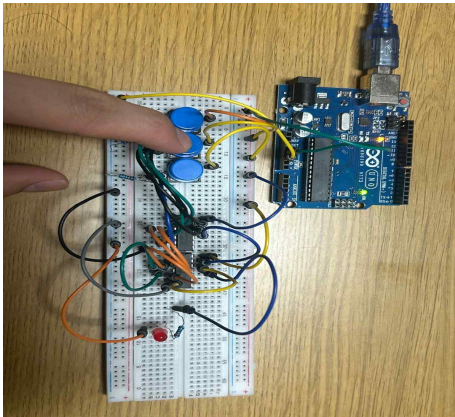
1. $A = 0, B = 0, C = 0$



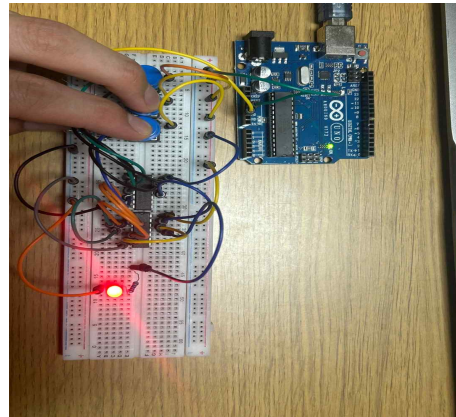
2. $A = 0, B = 0, C = 1$



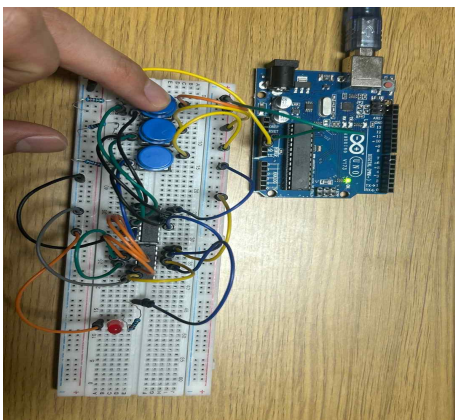
3. $A = 0, B = 1, C = 0$



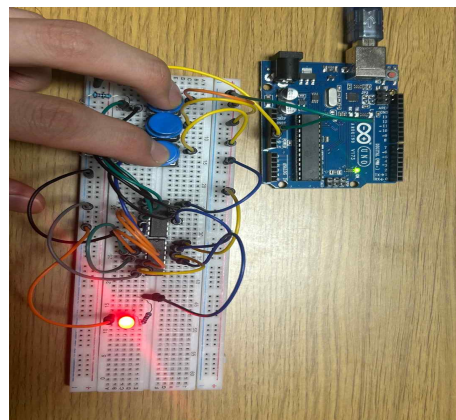
4. $A = 0, B = 1, C = 1$



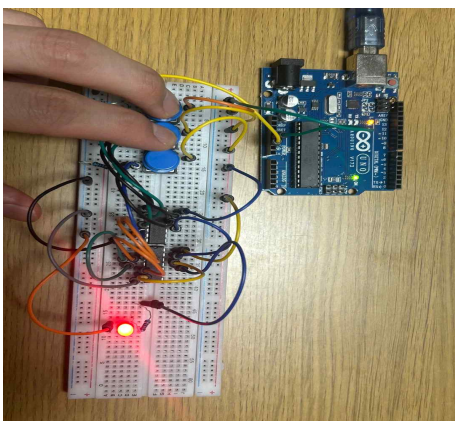
5. $A = 1, B = 0, C = 0$



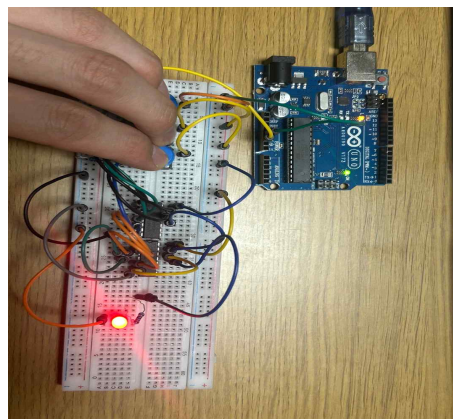
6. $A = 1, B = 0, C = 1$



7. $A = 1, B = 1, C = 0$



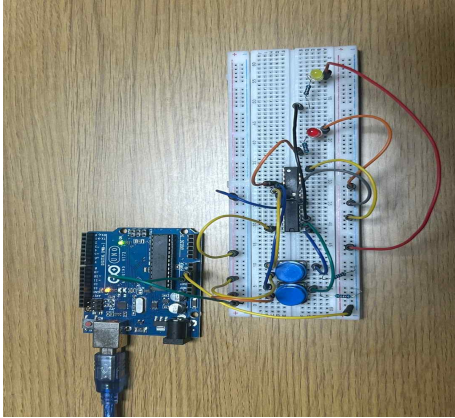
8. $A = 1, B = 1, C = 1$



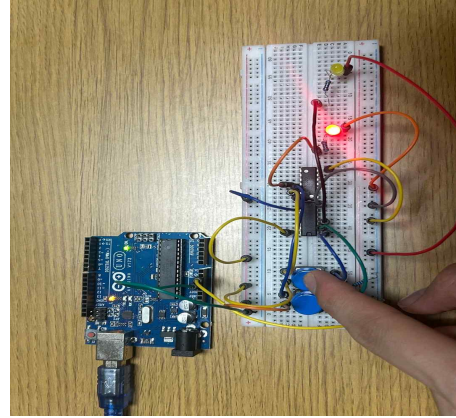
● 반가산기 회로

-(다이오드 순서 : SUM, CARRY)

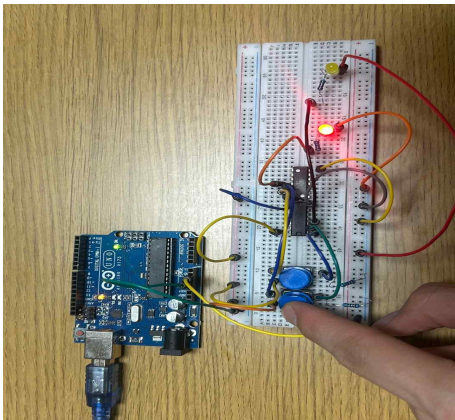
1. $A = 0, B = 0$



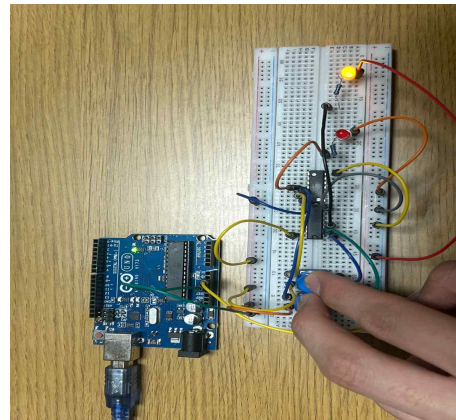
2. $A = 0, B = 1$



3. $A = 1, B = 0$



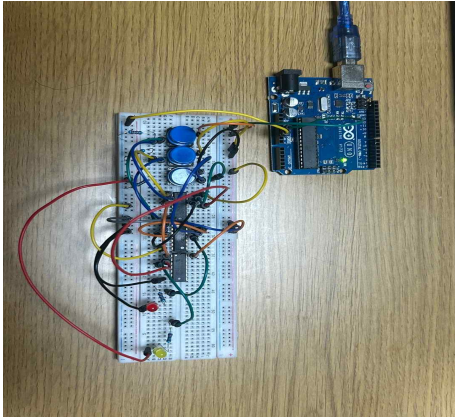
4. $A = 1, B = 1$



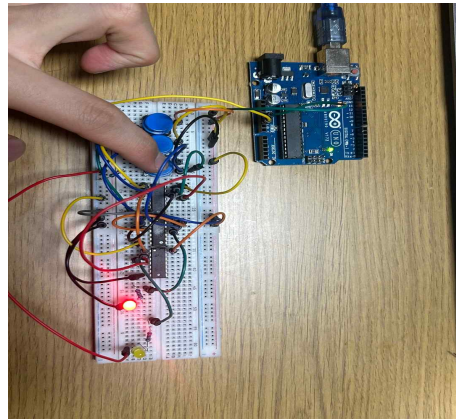
● 전가산기 회로

-(다이오드 순서 : SUM, CARRY)

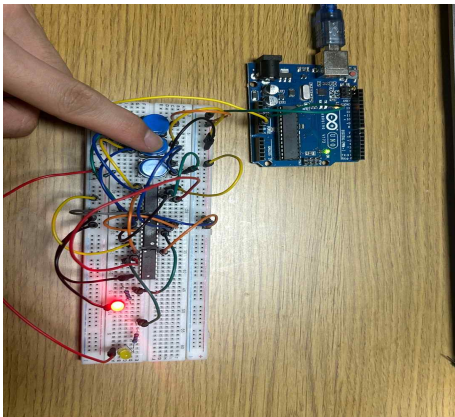
1. $A = 0, B = 0, C_0 = 0$



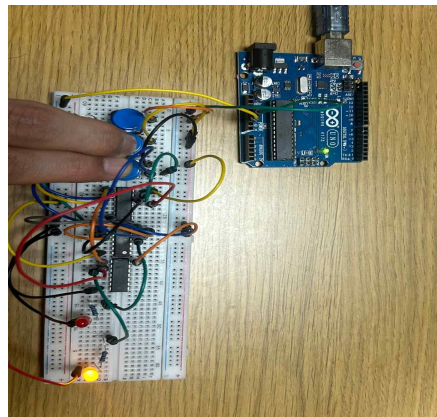
2. $A = 0, B = 0, C_0 = 1$



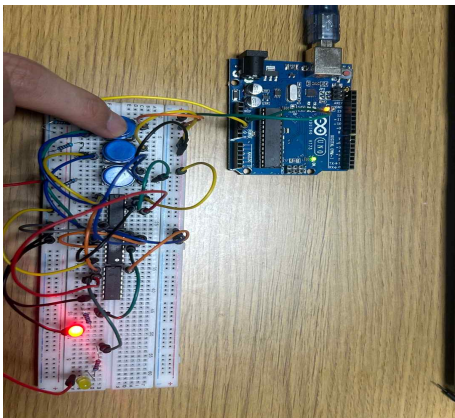
3. $A = 0, B = 1, C_0 = 0$



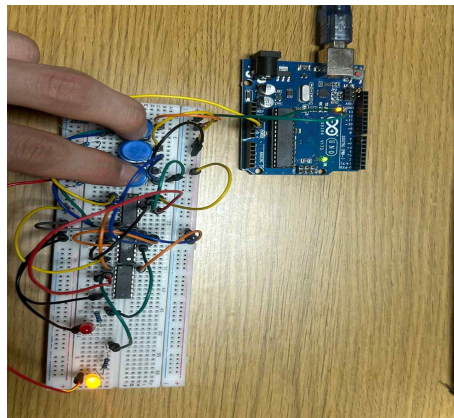
4. $A = 0, B = 1, C_0 = 1$



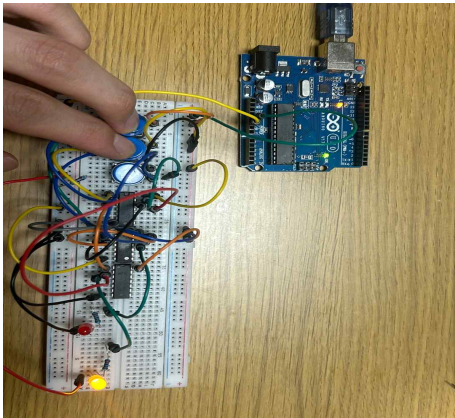
5. $A = 1, B = 0, C_0 = 0$



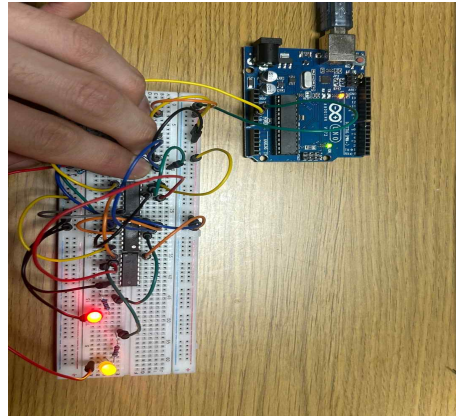
6. $A = 1, B = 0, C_0 = 1$



7. $A = 1, B = 1, C0 = 0$



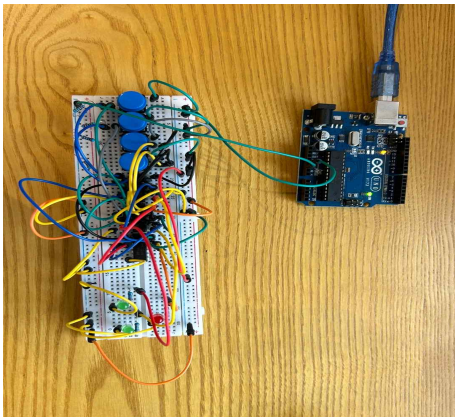
8. $A = 1, B = 1, C0 = 1$



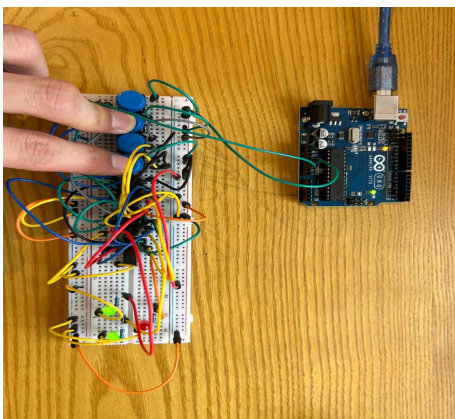
● 2비트 2진 가산기 회로

- (버튼순서 A1, B2, A2, B2 다이아드 순서 : C2(위), S1, S2)

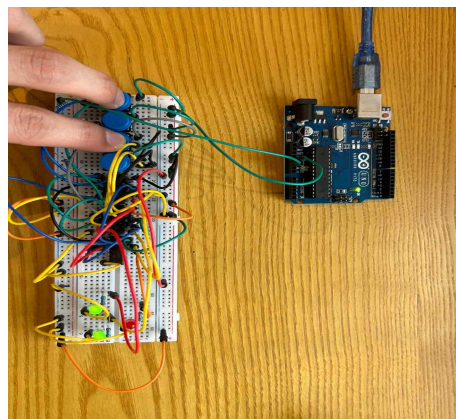
1. $A1=0, B1=0, A2=0, B2=0, C0=0$



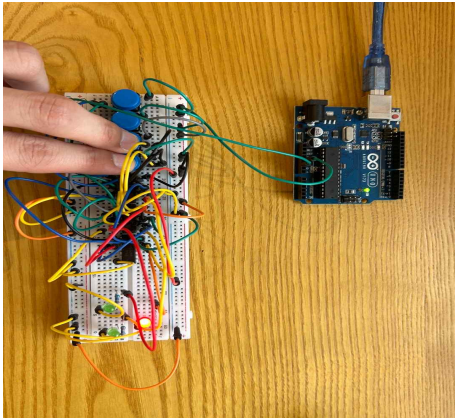
2. $A1=0, B1=1, A2=0, B2=1, C0=0$



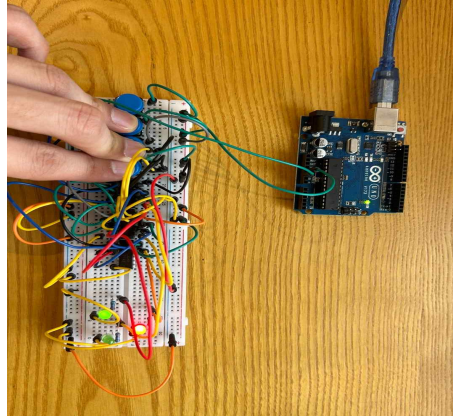
3. $A1=1, B1=0, A2=1, B2=0, C0=0$



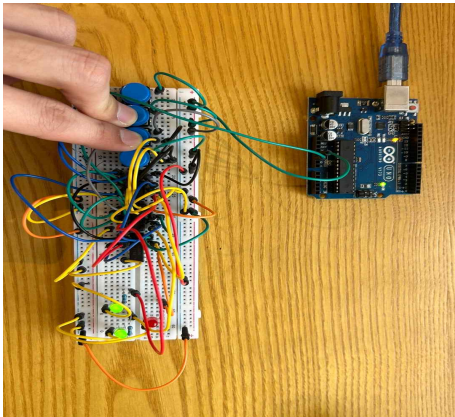
4. $A1=0, B1=0, A2=1, B2=1, C0=0$



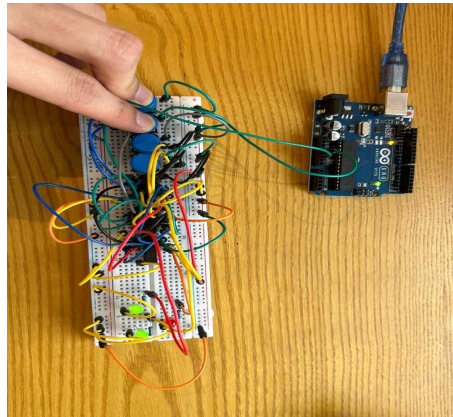
5. $A1=0, B1=1, A2=1, B2=1, C0=0$



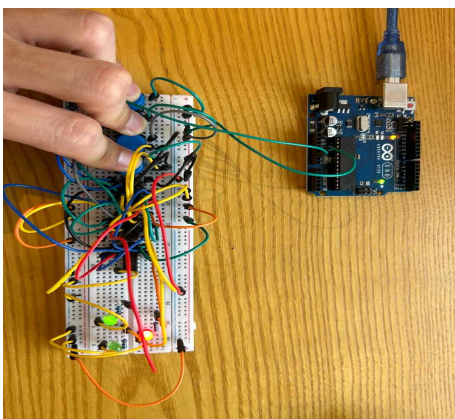
6. $A1=0, B1=1, A2=1, B2=0, C0=0$



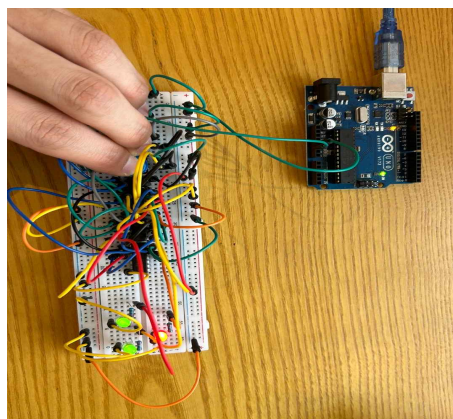
7. $A1=1, B1=1, A2=0, B2=0, C0=1$



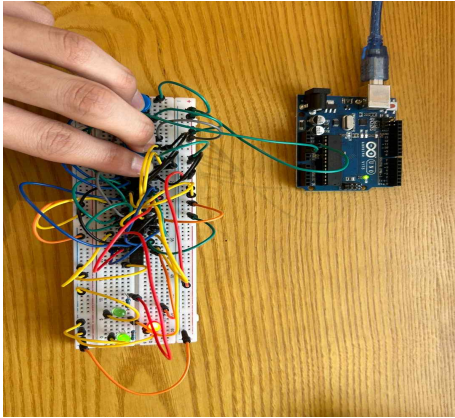
8. $A1=1, B1=1, A2=0, B2=1, C0=1$



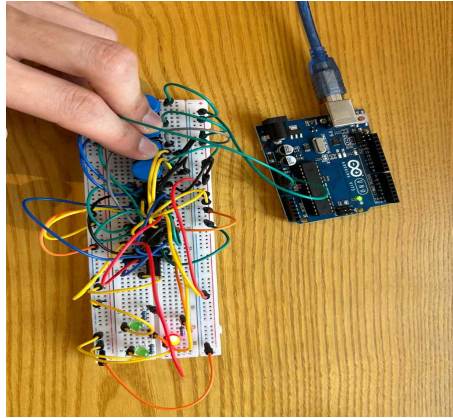
9. $A1=1, B1=1, A2=1, B2=1, C0=1$



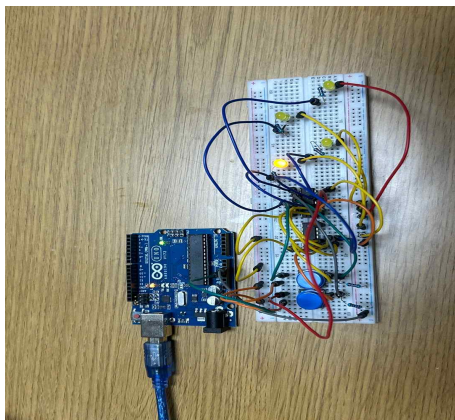
10. $A_1=1, B_1=0, A_2=1, B_2=1, C_0=1$



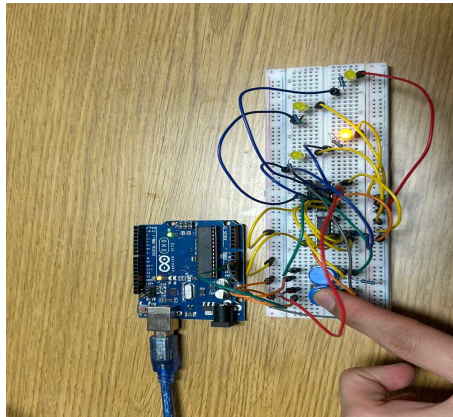
11. $A_1=0, B_1=1, A_2=1, B_2=0, C_0=1$



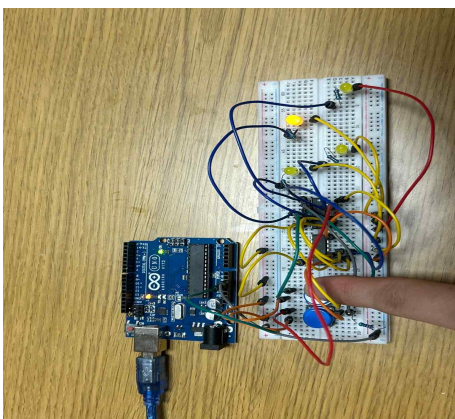
1. $A = 1, B = 0$



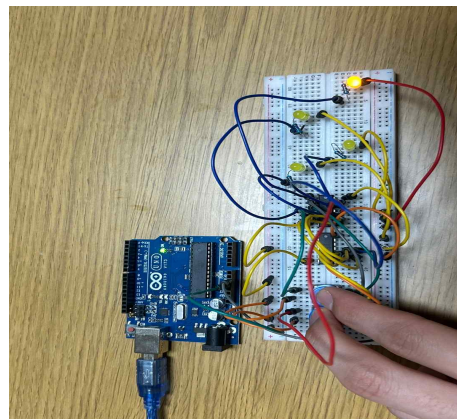
2. $A = 1, B = 1$



3. $A = 1, B = 0$

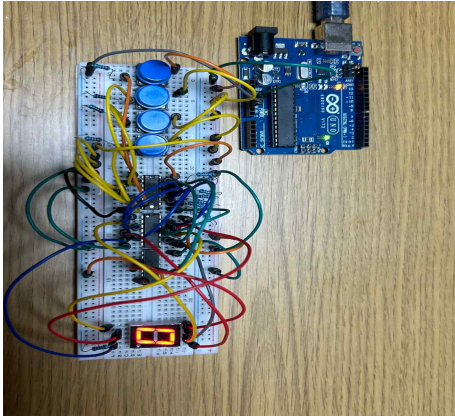


4. $A = 1, B = 1$

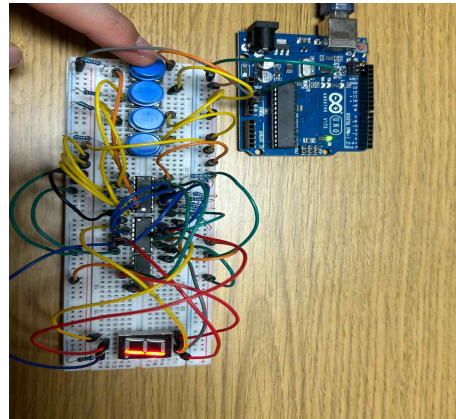


● 7 세그먼트 디코더 회로
(버튼순서 : A B C D)

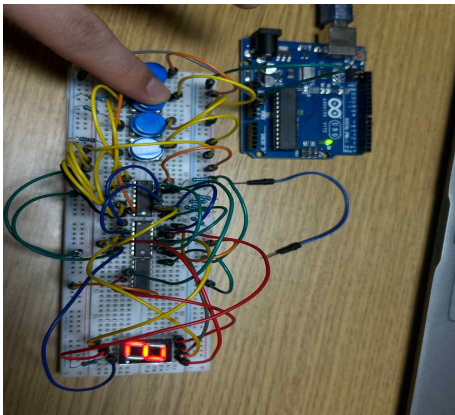
1. A=0, B=0, C=0, D=0



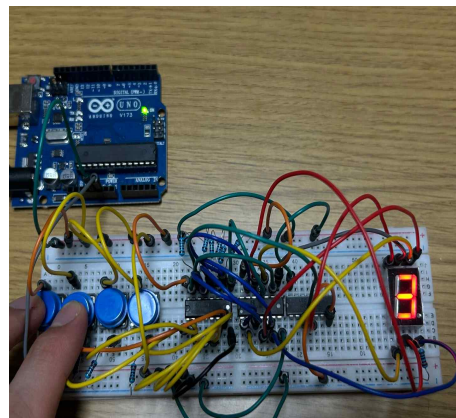
2. A=1, B=0, C=0, D=0



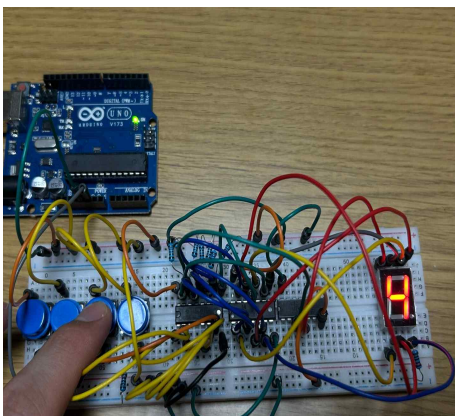
3. A=0, B=2, C=0, D=0



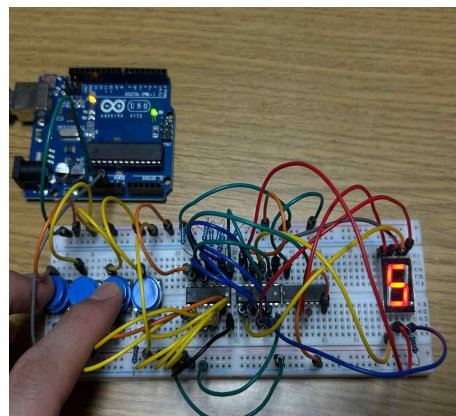
4. A=1, B=1, C=0, D=0



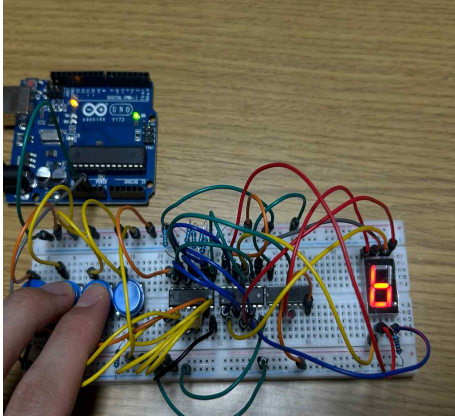
5. A=0, B=0, C=1, D=0



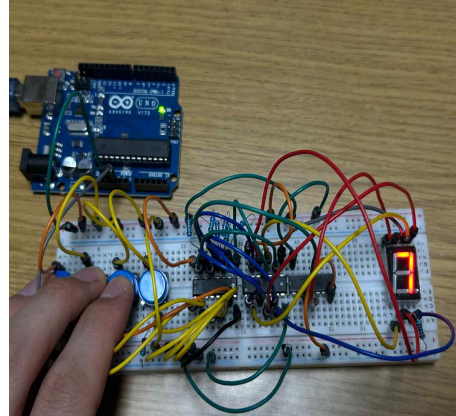
6. A=1, B=0, C=1, D=0



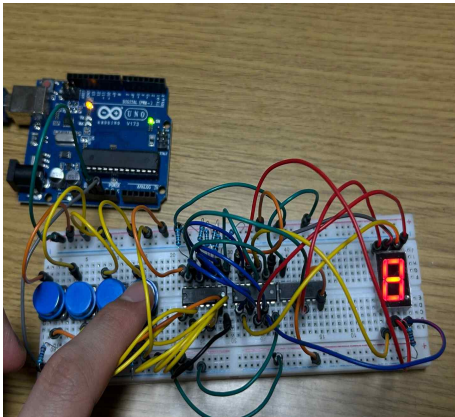
7. A=0, B=1, C=1, D=0



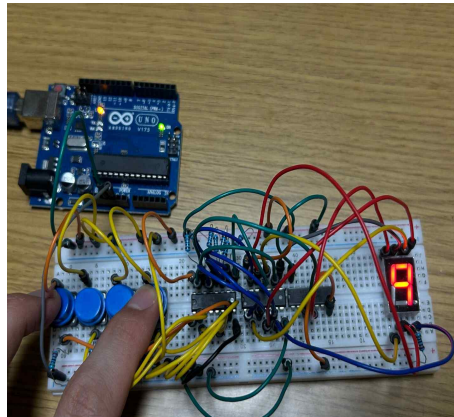
8. A=1, B=1, C=1, D=0



9. A=0, B=1, C=1, D=0



10. A=1, B=1, C=1, D=0



<실습의 요점, 검토 및 결과>

◆ NOT 게이트 실습- NOT 게이트는 부정 연산을 수행해 입력의 결과값을 바꿔준다.

A	X
0	1
1	0

◆ AND 게이트 및 NAND 게이트 실습- AND 게이트는 논리곱 연산을 수행한다. A B 0 0

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

- NAND 게이트는 AND 게이트와 반대로 부정 논리곱 연산을 수행한다.

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

◆ OR 및 NOR 게이트- OR 게이트는 논리합 연산을 수행한다

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

- NOR 게이트는 OR 게이트와 반대로 부정 논리합 연산을 수행한다

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

- ◆ XOR 게이트- XOR 게이트는 배타적 논리합 연산을 수행한다.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

- ◆ 분배법칙 (OR-AND / AND-OR)

- OR-AND 게이트와 AND-OR 게이트는 연산 순서는 다르지만 출력은 서로 같다.

- ◆ NAND 게이트를 이용한 OR 및 NOR 게이트

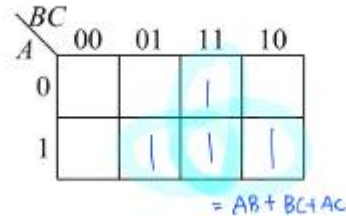
- NAND 게이트만으로 OR 게이트와 NOR 게이트를 표현할 수 있다.

- NAND 게이트로 모든 논리회로를 표현할 수 있기 때문에, NAND 게이트는 논리 게이트 중 가장 중요한 게이트라고 할 수 있다.

◆ 출력 X에 대한 카르노 맵과 논리회로

- 카르노 맵을 간략화하여 논리 회로를 설계할 수 있다

A	B	C	X	Y
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	0	
1	0	1	1	
1	1	0	1	
1	1	1	1	



A	B	C	X	Y
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

- 카르노 맵을 간략화시켜도 간략화하기 전과 출력 결과가 같다.

◆ 반가산기

- 2진수의 덧셈을 논리 회로로 표현할 수 있다.

◆ 전가산기

- 2진수의 덧셈 계산을 할 때 하위 비트의 수의 계산에서 올림수가 발생했을 때, 그에 대해서도 덧셈연산이 가능하다.

◆ 2비트 2진 가산기

- 전가산기 회로를 이어 붙여 모든 자리수에 대한 덧셈연산이 가능하다.

◆ 2X4 디코더

- 입력 신호의 개수가 n개 일 때 출력은 최대 2^n 가지로 나타낼 수 있다.

◆ 7 세그먼트 디코더

- 출력이 입력의 2배로 나올 수 있는 점을 활용해 16진수를 표현한 것이다

★ 느낀점

- 처음 아두이노를 받게 되었을때 가장 간단한 NOT게이트도 만들지 못 하였지만 기본적인 이론을 바탕으로 개념이 잡히면서 비교적 어려운 2비트 2진 가산기, 7 세그먼트 디코더 등도 혼자서 만들 수 있게 되었습니다. 특히 실력이 가장 많이 늘었을 때는 완성 했지만 이진표랑 다른 값을 출력했을 때 스위치부터가 아닌 맨 뒤부터 역순으로 돌아오며 하나 하나 입력과 출력을 체크하면서 원인을 찾을때였다. 선들이 복잡하여 쉽지 않았지만 완성하고 결과값이 정확하게 나왔을때의 뿌듯함이 정말 강하였다.